

CHAPTER ONE: INTRODUCTION

1.1 Background of the Study

Human Resource Management (HRM) is the organisational function responsible for the employment relationship across its whole span, from recruitment and onboarding through compensation, performance evaluation, and eventual separation. Every one of these activities generates data, and the volume of that data grows with headcount: personnel records, attendance logs, leave balances, appraisal scores, payroll computations, and supporting documents such as contracts, identity credentials, and curricula vitae. In smaller and mid-sized organisations this data has historically been managed through spreadsheets, paper files, and disconnected point solutions, each holding a partial and frequently inconsistent picture of the same employee.

The response to this fragmentation has been the progressive digitalisation of the HR function. Vu (2024) synthesises this trajectory and characterises it as movement across three levels: the operational use of digital tools for administrative tasks, strategic alignment in which those tools support an existing HR strategy, and strategic integration in which the technology begins to shape what that strategy is. The same review notes that the shift has not been uniformly beneficial, having been associated with new sources of work-related stress and, in some settings, weakened interpersonal communication. Digitalising HR is therefore not a neutral act of automation; the system encodes decisions about how HR processes are enforced and by whom.

Cloud computing supplied the infrastructure on which most of this digitalisation was built. Rather than purchasing and maintaining servers, an organisation rents compute, storage, and networking capacity and adjusts that capacity as demand changes. A systematic review of ninety studies of cloud adoption among small and medium-sized enterprises found operational efficiency to be the most frequently reported benefit, followed by cost savings and improved competitive positioning through cloud-enabled scalability (Mkhize et al., 2025). Within the HR domain specifically, Al-Mutawa and Al Mubarak (2025) found that smaller organisations value cloud HRM systems because they remove the upfront infrastructure investment while still delivering recruitment, payroll, and performance functionality that would otherwise require several separate on-premise tools. Zhao and Rabiei (2023) reached a comparable conclusion for payroll in particular, assessing the feasibility of migrating payroll management onto cloud infrastructure and identifying cost and maintenance relief as the dominant drivers.

The term "cloud-based" nonetheless conceals a significant architectural distinction. A conventional cloud HRIS typically runs on provisioned virtual machines or containers that must be sized, monitored, patched, and paid for whether or not they are serving a request at a given moment. Serverless computing represents a further step away from that model. In a serverless deployment the provider allocates compute only in response to an invocation, and billing follows execution time rather than allocated capacity. Wen et al. (2023), in a systematic review of the field, describe how this Function-as-a-Service model matured from

an experimental deployment pattern into a mainstream architectural choice, and identify event-driven invocation, near-instantaneous automatic scaling, and usage-based billing as the characteristics that separate it from the Infrastructure-as-a-Service and Platform-as-a-Service models preceding it. The trade-offs are equally well documented. Golec et al. (2024) catalogue cold-start latency as the most persistently studied limitation, together with the range of mitigation strategies developed in response, none of which removes the trade-off between latency, resource cost, and implementation complexity entirely. Scheduling behaviour under serverless execution has attracted similar attention (Ghorbian et al., 2024).

A further refinement combines serverless execution with edge computing, placing application code in data centres geographically close to end users rather than in a single centralised region. Raith et al. (2023) describe this combination as a response to a specific weakness of centralised serverless platforms, namely that network latency to a distant region can dominate total response time regardless of how quickly a function executes. Their review also identifies the open problems the combination introduces, including state and data consistency across geographically distributed execution sites and the constrained compute budgets typical of lightweight edge runtimes. Cloudflare Workers, the platform on which the present system is built, exemplifies this approach by executing application code inside lightweight V8 isolates that share an underlying process rather than requiring a container or virtual machine per invocation (Cloudflare, 2026a).

Two further considerations shape the present project. The first is statutory payroll compliance. In Nigeria, payroll carries obligations that go well beyond salary computation: progressive Pay-As-You-Earn (PAYE) income tax with a Consolidated Relief Allowance, mandatory pension contributions, National Housing Fund deductions, and employer-borne levies to the Nigeria Social Insurance Trust Fund and the Industrial Training Fund, each with its own remittance schedule and filing format. Reconciling these obligations by hand against attendance, leave, and loan data every pay cycle is laborious and is a recurring source of compliance risk. Falana et al. (2024) find, in the Nigerian context, that digital administration measurably improves compliance outcomes, which supports the case for encoding statutory rules into the payroll system itself rather than leaving them to manual application.

The second consideration is the arrival of large language models in HR software. Budhwar et al. (2023) set out both the promise and the hazard of generative artificial intelligence in HRM, and Dasaklis et al. (2026), reviewing the LLM-specific literature, map applications across recruitment, training, and performance evaluation while identifying governance and accountability as unresolved concerns. The hazard is concrete rather than abstract: prompt injection remains the highest-ranked vulnerability class for LLM applications, and excessive agency, in which a model is granted more functionality, permission, or autonomy than the task requires, is separately catalogued as a distinct risk (OWASP, 2025). An assistant with conversational access to salary records, disciplinary history, and payroll totals is precisely the case in which these risks matter, and the way such an assistant is wired into an existing authorisation model is therefore an engineering question in its own right.

This project brings these strands together. It designs and implements ZenHR, a multi-tenant Human Resource Management and payroll platform built entirely on serverless infrastructure: a React single-page frontend, a Cloudflare Workers backend, Cloudflare D1

for relational data, Cloudflare R2 for document storage, and Cloudflare Workers AI for the assistant module. The sections that follow state the problem this addresses, the aim and objectives adopted in response, the boundary of the work, and the methodology followed.

1.2 Statement of the Problem

Organisations that manage HR and payroll through disconnected tools, or through a conventional provisioned-server HRIS, encounter a set of related problems that this project addresses directly.

Data fragmentation across the employee lifecycle. Employee records, attendance, leave balances, salary configuration, and performance history commonly reside in separate systems, so every payroll run begins with manual reconciliation. Destriani et al. (2024), reviewing HRMS implementation, identify integration difficulty and limited in-house technical capability as recurring obstacles, both of which are aggravated when the system of record is split across tools that were never designed to communicate.

Statutory compliance risk in payroll. Nigerian PAYE bands, Consolidated Relief Allowance treatment, pension and housing-fund deductions, and employer levies are each easy to compute incorrectly by hand and easy to remit late without a tracked calendar. A payroll process that recomputes these figures from re-keyed data, rather than from live attendance and loan records, multiplies the opportunities for error, and errors here carry regulatory as well as financial consequences.

Fixed-capacity cost under variable demand. HR activity is bursty rather than constant. A month-end payroll run, a recruitment drive, and an appraisal cycle each produce a spike in load, while the system may serve only a trickle of requests between them. Provisioned virtual machines or containers must be sized for the peak and billed for the trough. Where a platform serves several tenant organisations, this compounds rather than merely repeats: shared provisioned infrastructure must be sized for the combined peak of all tenants, forcing a choice between over-provisioning and accepting that one tenant's load can degrade service for the others (Hattab & Belalem, 2023).

Operational overhead falling on those least able to carry it. Provisioned infrastructure requires someone to size instances, patch operating systems, configure scaling rules, and monitor capacity. The smaller organisations that stand to gain most from a consolidated HR platform are the same ones least equipped to staff that work (Al-Mutawa & Al Mubarak, 2025; Mkhize et al., 2025).

Tenant isolation and role-appropriate visibility. A platform serving multiple client organisations must guarantee that no tenant's data is reachable from another tenant's session, at every query rather than only at the interface layer. Within a tenant, an administrator, a line manager, a payroll officer, and an ordinary employee each require a different and correctly bounded view of the same underlying records.

Weak auditability of sensitive changes. Salary edits, role changes, and access grants are consequential actions that spreadsheet-based workflows record poorly or not at all, leaving no reliable trail when a figure is later disputed.

Unresolved access-control questions when an LLM is introduced. Conversational assistants are being added to HR software rapidly (Dasaklis et al., 2026), but a model with broad data access inherits none of the application's authorisation logic by default, and the dominant vulnerability classes for such systems concern exactly this gap (OWASP, 2025). How to expose a language model to sensitive HR data without weakening the guarantees the surrounding system already provides is not settled in the literature.

Serverless, edge-based architecture addresses the third and fourth of these problems by construction, since compute is allocated per request and there is no idle instance to pay for or maintain. The architecture has, however, been studied predominantly in general-purpose and infrastructure-level contexts, such as data pipelines, machine-learning inference, and high-performance computing workloads (Ghorbian et al., 2024; Wen et al., 2023), rather than applied and evaluated against the mixed operational profile of an HRMS: relational transactional data, unstructured document storage, multi-step approval workflows, computationally non-trivial statutory payroll, and strict tenant isolation, all in one system.

This project therefore addresses the problem of designing and implementing a multi-tenant HRMS and payroll platform on a fully serverless architecture that eliminates fixed infrastructure cost and manual scaling overhead, encodes Nigerian statutory payroll rules directly into the computation path, enforces tenant and role isolation at the data-access layer, and integrates a language-model assistant without relaxing any of those guarantees.

1.3 Aim and Objectives

Aim

The aim of this project is to design and implement ZenHR, a scalable multi-tenant Human Resource Management System with an integrated statutory payroll engine, built entirely on serverless cloud architecture using Cloudflare Workers, D1, R2, and Pages with a React.js frontend, in order to eliminate the fixed infrastructure cost and operational overhead of conventional provisioned deployments while maintaining strict tenant isolation, role-scoped access, and auditable statutory compliance.

Objectives

1. **Review the literature** on cloud-based and serverless architecture, multi-tenant SaaS design, HR information systems, payroll automation, and language-model integration in HRM, in order to establish current approaches and confirm the specific gap this project addresses.
2. **Design a relational data model and multi-tenant isolation strategy** covering the full employee lifecycle across HR, payroll, performance, recruitment, and benefits domains, implemented on Cloudflare D1's SQLite-based engine, with a tenant discriminator applied consistently at the data-access layer.
3. **Implement token-based authentication and layered authorisation**, comprising signed JSON Web Token authentication, coarse-grained role gating across six roles, and an additive fine-grained permission layer, such that every data access is scoped to the correct tenant and the correct role.

4. **Implement the core transactional HR modules:** workforce and organisational records, onboarding and offboarding transitions, attendance and overtime, and leave application and approval.
5. **Implement a statutory payroll engine** that computes gross-to-net pay per employee from live system data, applying salary proration for mid-month hires, the Nigerian progressive PAYE bands with Consolidated Relief Allowance, pension, National Housing Fund, Nigeria Social Insurance Trust Fund and Industrial Training Fund treatment, and loan amortisation, and that generates a tracked remittance calendar and scheme-specific export files.
6. **Implement the extended HR modules:** recruitment and applicant tracking with a public careers page, performance management including review cycles and 360-degree feedback, benefits and wellness administration, asset tracking, learning management, employee surveys, and internal support ticketing.
7. **Implement a role-scoped AI assistant** on Cloudflare Workers AI in which the model is given a fixed set of read-only tools rather than direct data access, each tool independently re-derives the caller's authorisation from the verified session rather than trusting the model's arguments, and every query is written to a persistent access log.
8. **Evaluate the system** through automated backend testing, empirical response-time measurement of core operations across modules, and an analytical comparison of the platform's usage-based cost model against the fixed-cost profile of an equivalent provisioned-server deployment.

Each objective maps forward to a specific part of the report. Objective 1 is discharged in Chapter Two; Objectives 2 and 3 in Chapter Three; Objectives 4 through 7 in Chapter Four's implementation sections; and Objective 8 in Chapter Four's testing and results sections.

1.4 Scope of the Study

The project covers the design and implementation of a working multi-tenant HRMS and payroll platform across the following functional areas.

Tenancy, identity, and access. Provisioning of client organisations as isolated tenants; company profile and settings; organisational structure through departments and locations; six fixed user roles with an optional custom permission matrix layered over them; JWT-based authentication with hashed credential storage; and API key issuance.

Workforce records. Employee directory and profile management, emergency contacts, a per-employee document store, department and location assignment, and a per-employee audit history view.

Onboarding and offboarding. Both are modelled as tracked transitions composed of categorised task checklists across HR, IT, Finance, and Administration, with overall stage derived from task completion. Offboarding additionally records separation reason, handover recipient, and exit-interview scheduling.

Attendance and time tracking. Clock-in and clock-out with optional geolocation capture, multiple sessions per day, computed work hours, and a separate overtime request and

approval workflow. Punctuality classification is driven by company-configured start time, grace period, and working-day calendar.

Leave management. Typed leave entitlements, employee-initiated applications with optional attachments, manager approval with comments, and balance reporting derived from approved requests.

Recruitment and applicant tracking. Job requisitions with an approval workflow, a candidate pipeline through screening, interview, offer, and hire, per-interviewer scorecards supporting panel decisions, a per-candidate timeline of every stage transition, and a public unauthenticated careers page through which external applicants can apply.

Payroll and statutory compliance. Payroll configuration, per-employee gross-to-net computation with proration and salary component splitting, Nigerian PAYE with editable progressive bands, pension, National Housing Fund, Nigeria Social Insurance Trust Fund and Industrial Training Fund handling with correct employee-versus-employer attribution, loan deduction and amortisation, a run lifecycle from preview through submission, approval, and payment marking, automatic generation of remittance compliance tasks with statutory due dates, and export of bank disbursement and scheme-specific remittance files.

Performance and development. Review cycles with named stages, employee self-assessments, manager reviews, peer and upward reviews with nomination control, goals with progress tracking and parent-goal alignment, peer recognition, and training records.

Benefits, wellness, and learning. Benefit plan catalogue with employer and employee cost split, enrolment and dependent registration, wellness programmes with participant progress, a shared claims workflow, and a lightweight learning management module with course assignment and progress tracking.

Assets, surveys, and support. Company asset registry with assignment and condition tracking, employee pulse and eNPS surveys with optional anonymity, and an internal helpdesk with threaded ticket messaging.

Administration and audit. A control centre covering security settings, operational calendar, custom roles, email templates, integration catalogue, automation catalogue, full tenant data export, and a searchable, exportable audit log of sensitive administrative actions.

AI assistant. A conversational assistant over a fixed set of read-only tools, with authorisation re-derived inside each tool, a bounded agent loop, and persistent query logging.

Evaluation. Automated unit and integration testing of the backend, empirical response-time measurement of representative operations, and analytical cost comparison against a provisioned-server baseline.

Limitations and boundaries

1. **No payment disbursement.** The system computes net pay and generates bank transfer and statutory remittance files, but does not integrate with a banking or payment rail and therefore does not move funds.
2. **No concurrent or burst load testing.** Access to a load-generation environment representative of multi-tenant peak traffic was not available. Scalability is therefore argued analytically from the platform's per-request execution model and supported by single-request response-time measurement, rather than demonstrated empirically under simulated high-volume traffic.
3. **Third-party integrations are limited.** Of the integration catalogue exposed in the control centre, only the Slack webhook connection performs a live handshake; the remainder are configuration entries without an execution engine behind them. The automation workflow catalogue is likewise definitional rather than executing.
4. **No formal security audit or penetration test.** Security controls are validated through the project's own functional and unit testing. The audit, retention, and access-control design is informed by recognised information-security control principles but the project neither pursues nor claims formal certification against any standard.
5. **No end-to-end testing.** Automated test coverage is implemented for both the backend and the frontend, but each suite tests its own layer against a mocked dependency: the backend suite mocks the database, and the frontend suite mocks the API client. No test drives the user interface against a running backend, and no test runs against the deployed platform.
6. **No continuous integration or deployment pipeline.** Deployment is performed manually through the platform CLI.
7. **Single active performance review cycle.** The performance module assumes one company-wide active review cycle at a time and does not support overlapping or department-scoped cycles.
8. **Simplified remittance cadence.** Compliance tasks for the employer-borne levies are generated each payroll run rather than modelling each scheme's true statutory filing frequency, favouring uniform tracking over exact cadence.
9. **No legacy data migration tooling.** The system does not provide import utilities for HR records held in an existing or third-party system.
10. **Read-only AI tooling.** The assistant is restricted to retrieval; it cannot create, modify, or delete records, and write-capable tooling is deliberately excluded.

1.5 Methodology

The project follows two related methodologies: a research methodology governing how the problem was investigated, and a software methodology governing how the artefact was built. Both are stated here at a high level; Chapter Three develops the software methodology into concrete design.

1.5.1 Research Methodology

The research methodology adopted is design science research, which suits a project whose central contribution is a functioning artefact rather than a purely theoretical result. The following steps were followed.

1. **Problem identification.** Establishing, through the background and problem statement above, the gap between conventional provisioned HRIS deployments and the operational demands of a multi-tenant, statutorily-compliant HR platform.
2. **Literature review.** Surveying cloud, serverless, multi-tenancy, HR information systems, payroll automation, and LLM-in-HRM literature (Chapter Two) to confirm that gap and to ground the architectural choices made in response.
3. **Design and development.** Translating the objectives in Section 1.3 into a system architecture, data model, and working application on the Cloudflare platform (Chapters Three and Four).
4. **Demonstration.** Implementing each module to a functioning standard sufficient to exercise its core workflows end to end.
5. **Evaluation.** Assessing the resulting system against the aim through automated testing, response-time measurement, and analytical cost comparison (Chapter Four).
6. **Communication.** Reporting the design, implementation, and findings in this report and in the project defence.

1.5.2 Software Methodology

The system was developed using an incremental, feature-driven Agile approach organised around vertical slices. Rather than building horizontal layers separately, each development increment delivered one complete slice through the stack: the database schema for a feature, the service holding its business logic, the controller and route exposing it, the frontend screen consuming it, and the tests covering it.

This suits the project for two reasons. The feature set spans a number of largely independent modules, which lend themselves to being built and tested one at a time rather than all at once. A serverless architecture also supports incremental delivery well, since individual routes and tables can be added and deployed without disturbing a monolithic server build.

The resulting development sequence was risk-first: authentication, multi-tenancy, and the employee record were built and hardened before any dependent module, followed by the core transactional modules of attendance and leave, then payroll, then progressively more specialised modules. Section 4.2 reports how each module was built under this cycle, and Section 4.3 reports the testing applied at each increment.

1.6 Significance of the Study

Practical significance for organisations. A working multi-tenant HRMS on serverless infrastructure demonstrates a concrete alternative to the provisioned-server model underlying most cloud HRIS products. For smaller organisations, the segment least equipped to absorb capacity planning and maintenance overhead (Al-Mutawa & Al Mubarak, 2025; Mkhize et al., 2025), a platform that scales per request and bills per use rather than per idle server hour lowers the barrier to adopting a consolidated HR system covering the whole employee lifecycle in one place.

Significance for payroll compliance practice. The payroll engine encodes Nigerian statutory rules directly into the computation path and derives every figure from live

employee, attendance, and loan data rather than from re-keyed input. It also generates a tracked remittance calendar with real statutory due dates, converting a manual reminder process into a system-enforced one. Given evidence that digital tax administration improves compliance outcomes in the Nigerian context (Falana et al., 2024), a payroll system that makes statutory computation transparent and auditable rather than opaque is of direct practical value, particularly for organisations without dedicated payroll specialists.

Contribution to the body of knowledge on serverless architecture. Serverless computing has been studied extensively at the infrastructure level but considerably less as applied to document-heavy, workflow-driven, multi-tenant business systems (Ghorbian et al., 2024; Wen et al., 2023). This project contributes an evaluated case study of how a serverless edge architecture performs against the specific demands of such a system, including tenant isolation, workflow orchestration, computationally non-trivial payroll, and mixed relational and object storage. It gives future researchers and developers a concrete reference point for systems of this shape.

Contribution to the integration of language models into sensitive systems. The assistant module demonstrates an auditable pattern for exposing a language model to HR and payroll data: the model is given tools rather than data access, authorisation is re-derived inside each tool implementation from the caller's verified identity rather than inferred from the model's arguments, the agent loop is bounded, and every query is logged. Given that excessive agency and prompt injection are the leading catalogued risks for LLM applications (OWASP, 2025), and that governance remains an open concern in the LLM-in-HRM literature (Budhwar et al., 2023; Dasaklis et al., 2026), a documented implementation pattern that preserves the surrounding system's access-control guarantees is a contribution beyond the HR domain itself.

1.7 Project Outline

This report is organised into five chapters.

Chapter One introduces the project, presenting the background to human resource management, cloud and serverless computing, statutory payroll compliance, and language-model integration in HR software; stating the problem addressed; setting out the aim and objectives; defining the scope and its limitations; outlining the research and software methodologies followed; and discussing the significance of the work.

Chapter Two reviews the relevant literature, tracing the development of HR information systems, the progression from on-premise through cloud to serverless and edge-native architecture, the multi-tenancy design problem, payroll and statutory compliance automation, and the emerging literature on language models in HRM. It reviews existing HR management systems, both academic and commercial, and establishes the specific gap this project addresses.

Chapter Three presents the system methodology and design, expanding the high-level methodology of Chapter One into a concrete system architecture, conceptual framework, functional and non-functional requirements, use-case, data-flow and entity-relationship diagrams, the multi-tenant database design on Cloudflare D1, the layered request pipeline,

interface design, and the security design underlying authentication, authorisation, audit, and the AI assistant's tool boundary.

Chapter Four describes the implementation, covering the development environment and toolchain, then working through each module in turn: tenancy and access control, workforce records, onboarding and offboarding transitions, attendance, leave, recruitment and applicant tracking, the statutory payroll engine, performance management, benefits and wellness, the extended modules, and the AI assistant. It then reports the automated testing carried out, the response-time measurements taken, and the cost analysis used to evaluate scalability and cost efficiency against the aim.

Chapter Five concludes the project, summarising the study and its findings, assessing how far the aim and objectives were achieved, discussing observations and problems encountered during implementation, and offering recommendations and suggestions for future work, including areas placed outside this project's scope such as concurrent load testing, payment disbursement, and formal security certification.

CHAPTER TWO: REVIEW OF RELATED LITERATURE

2.1 Introduction

This chapter reviews the literature relevant to the design and implementation of a multi-tenant Human Resource Management System with an integrated statutory payroll engine, built on serverless cloud architecture. The review is organised to move from the general to the specific, and within each theme from earlier to more recent work.

It begins by establishing what HR management systems are and how the concept has developed (Section 2.2), then examines the traditional and fragmented approaches that motivated their adoption (Section 2.3). It reviews cloud computing as the infrastructure response to those limitations (Section 2.4), then narrows to serverless and edge-native architecture specifically (Section 2.5). Because the system under development serves multiple client organisations, Section 2.6 reviews the multi-tenancy and tenant-isolation literature as a distinct design problem. The review then returns to the HR domain, examining how cloud technology has been applied to HR systems (Section 2.7), how payroll automation and statutory compliance have been treated (Section 2.8), and what the emerging literature says about artificial intelligence and large language models in HR (Section 2.9). Section 2.10 reviews existing HR management systems, academic and commercial, against the feature set and architecture this project adopts. Section 2.11 consolidates the limitations these systems share, and Section 2.12 states the resulting research gap and how the aim and objectives in Section 1.3 respond to it.

Consistent with departmental guidance on literature currency, the review draws on sources published within approximately the last three to five years. Where an older source is used, it is used to establish historical context and is identified as such rather than presented as evidence of the current state of the art.

{FIGURE PLACEHOLDER — Figure 2.1: A horizontal timeline diagram showing the architectural evolution of HR systems in five stages, left to right: (1) Manual and paper-based records; (2) Standalone desktop and mainframe payroll software; (3) On-premise networked HRIS; (4) Cloud-hosted HRIS on provisioned virtual machines and containers; (5) Serverless, edge-native multi-tenant HRMS. Beneath each stage, note its defining limitation, with the final stage annotated as the position this project occupies. Caption goes BELOW the figure.}

Figure 2.1: Architectural evolution of human resource management systems

2.2 Human Resource Management Systems: Concept and Development

Human Resource Management is the organisational function responsible for the employment relationship across recruitment, onboarding, compensation, performance management, development, and separation. As organisations sought to reduce the administrative load this function carries, they increasingly relied on dedicated information systems, producing the field variously described in the literature as electronic HRM, and more recently as digital HRM.

A distinction that remains useful for categorising systems separates a Human Resource Information System (HRIS), which is centred on storing and organising employee data such as personal records, compensation history, and organisational structure, from a Human Resource Management System (HRMS), which extends that data foundation with the workflows and services required to act on it: payroll processing, benefits administration, time and attendance, recruitment, and performance management. The present project adopts the broader HRMS framing throughout, consistent with an objective set that spans the whole employee lifecycle rather than record-keeping alone.

Vu (2024) synthesises the digital HRM literature and identifies digitalisation of HR practice at three levels. At the operational level, digital tools support administrative tasks such as recruitment and compensation. At the level of strategic alignment, those tools support the execution of an existing HR strategy. At the level of strategic integration, digital technologies begin to shape what the HR strategy itself is. The same review records that this progression has produced mixed rather than uniformly positive outcomes: gains in HR efficiency have been accompanied by new sources of work-related stress and, in some organisational contexts, weaker interpersonal communication.

This finding matters for a project of this kind. A system that encodes onboarding sequencing, attendance penalties, performance scoring, and payroll deductions is not a passive repository. It structures how HR decisions are made and enforced, and its design carries organisational consequences beyond the technical architecture supporting it. Where this project automates a workflow, the design intent is to remove clerical error and provide an auditable record, not to remove human judgement from decisions that require it.

Maqueira Marín et al. (2022), reviewing the intersection of cloud computing and HRM, reach a related conclusion from the technology side, observing that the literature has concentrated on adoption drivers and organisational outcomes rather than on how such systems should be architected. Taken together, this body of work establishes HRMS as a well-understood concept at the level of what such systems should do, while leaving comparatively less settled ground on how they should be built to deliver that functionality at scale and at acceptable cost. That architectural question is the specific angle this project investigates, and Sections 2.3 through 2.6 trace it forward.

2.3 Traditional and Fragmented HR Management Approaches

Before dedicated information systems became widespread, HR administration was carried out primarily as a manual, paper-based function. Personnel files, leave registers, and payroll calculations were maintained by hand or, where computerised at all, on isolated systems

used almost exclusively for payroll rather than broader HR administration. This early period treated HR data as a back-office record-keeping obligation rather than a source of organisational insight, and the systems that existed were not designed to communicate with one another or with the rest of the business. This account is included as historical background rather than as evidence of current practice.

As networked enterprise software became available, organisations adopted standalone HR packages, initially for payroll and benefits and gradually extending into recruitment tracking and reporting. The progression was uneven: the most data-intensive and rule-bound processes were digitised first, while less structured functions such as performance management and succession planning followed more slowly, since they required organisational judgement that is harder to encode.

Several structural weaknesses persisted through this period, and they remain observable in organisations that rely on spreadsheets and disconnected point solutions today. Destriani et al. (2024), in a systematic review of HRMS implementation, identify integration difficulty and limited in-house technical capability as recurring obstacles, both of which are direct consequences of fragmentation. Four weaknesses in particular are relevant to the design of the present system.

Data fragmentation. Where payroll, attendance, recruitment, and performance tools are implemented as separate systems rather than a unified platform, employee data ends up scattered across disconnected databases, spreadsheets, and paper files, with no single authoritative record spanning an employee's full lifecycle. Every downstream process that depends on more than one of these sources begins with manual reconciliation.

Manual, error-prone processing. Payroll computation, leave balance tracking, and compliance verification carried out by hand or through spreadsheet logic are vulnerable to human error at a volume that grows directly with headcount. In payroll specifically, the consequences of that error are financial and regulatory rather than merely administrative.

Fixed, on-premise infrastructure. Where systems were computerised, they typically ran on hardware the organisation owned and maintained, so scaling to a larger workforce required purchasing and provisioning additional physical capacity. This is a direct precursor to the fixed-capacity problem identified in cloud deployments in Section 1.2, with an even higher capital cost and a slower response to changing demand.

Weak cross-functional workflow support. Processes that inherently require coordination across departments, such as an offboarding sequence needing sign-off from a line manager, IT, finance, and HR, had no system to enforce that coordination. They relied on manual follow-up, physical paperwork routed between offices, or informal email threads leaving no audit trail.

The last of these is particularly relevant to the present project, since the onboarding and offboarding transition modules are a direct response to it: the required sign-offs are encoded as task records whose completion drives the overall case state, rather than being tracked by a person remembering to chase them.

2.4 Cloud-Based Information Systems

Cloud computing delivers computing resources, including servers, storage, databases, and networking, over the internet on demand rather than through infrastructure the consuming organisation owns and maintains. The model is conventionally characterised by on-demand self-service, broad network access, resource pooling across multiple consumers, rapid elasticity, and measured, pay-per-use billing. Cloud services are delivered through three established models: Infrastructure as a Service, in which the provider supplies raw compute, storage, and networking that the consumer configures; Platform as a Service, in which the provider additionally manages the runtime environment; and Software as a Service, in which the consumer accesses a complete, provider-managed application, as is the case with most commercial HRIS products reviewed in Section 2.10.

{TABLE PLACEHOLDER — Table 2.1: A comparison table with five columns (Delivery model; What the provider manages; What the consumer manages; Billing basis; Representative example) and four rows (IaaS; PaaS; SaaS; FaaS/Serverless). The FaaS row should show the provider managing the runtime, scaling, and capacity, the consumer managing only function code, billing tied to execution time rather than allocated capacity, and Cloudflare Workers as the example. Table title goes ABOVE the table.}

Table 2.1: Cloud service delivery models compared

A systematic review of ninety peer-reviewed studies of cloud adoption among small and medium-sized enterprises found operational efficiency to be the most frequently reported benefit, cited in roughly four out of five of the studies examined, followed by cost savings and improved competitive positioning arising from cloud-enabled scalability (Mkhize et al., 2025). The same review found that concerns over data security and privacy remained the most cited barrier to adoption, alongside the initial cost and complexity of migration. Cloud adoption is therefore broadly beneficial but not without trade-offs that a system's design must account for rather than assume away.

Two findings from this literature bear directly on the present project. First, the SME-focused evidence supports what Section 1.2 argues about conventional HRIS deployment: organisations with limited technical and financial resources gain the most in relative terms from cloud adoption, because it removes the upfront capital cost and the in-house maintenance burden of on-premise infrastructure (Mkhize et al., 2025). Second, and more consequentially for this project's architectural angle, the evidence base is drawn almost entirely from studies of conventional IaaS, PaaS, and SaaS deployments, that is, from the model in which an organisation rents a virtual machine or a managed application. It does not extend to the further step this project takes toward serverless, edge-native infrastructure. This is not a criticism of that literature; it marks where the general cloud evidence base ends and where the more specific territory covered in Section 2.5 begins.

2.5 Serverless and Edge-Native Computing Architecture

Serverless computing is a cloud execution model in which the provider allocates compute resources only in response to a specific invocation, rather than running a continuously provisioned server or virtual machine on the consumer's behalf. Wen et al. (2023), in a systematic review of the field, describe how this Function-as-a-Service model matured from an experimental deployment pattern into a mainstream architectural choice, precisely because it transfers responsibility for capacity planning, scaling, and server maintenance from the application developer to the platform. Their review identifies event-driven invocation, automatic and near-instantaneous scaling, and billing tied to actual execution time rather than allocated capacity as the characteristics that distinguish serverless computing from the models discussed in Section 2.4. Shafiei et al. (2022) survey the same terrain from an applications perspective and reach a comparable characterisation, while cataloguing the classes of workload for which the model has proved suitable.

The economic argument for the model has itself been examined empirically. Liu and Niu (2023) analyse the cost structure of serverless execution and find that whether it produces a saving depends on workload shape rather than being universally favourable: workloads with low duty cycles and irregular arrival patterns benefit substantially, whereas sustained high-utilisation workloads may not. This is directly relevant to the present project's cost argument, since HR workloads are characteristically bursty. Payroll runs, recruitment drives, and appraisal cycles concentrate activity into short periods, with comparatively little traffic between them, which is the workload profile for which the serverless billing model is most advantageous. Section 4.4 returns to this point with a concrete comparison rather than a general claim.

The architecture carries well-documented technical trade-offs, and the literature is candid about them. The most consistently studied is cold-start latency: because a function's execution environment is not kept permanently running, an invocation arriving after a period of inactivity incurs a delay while that environment is initialised. Golec et al. (2024), in a systematic review and taxonomy of this problem, catalogue the range of mitigation strategies developed in response, from keeping functions artificially warm through periodic invocation to lighter-weight execution environments that reduce initialisation cost, while noting that no single strategy eliminates the trade-off entirely; each balances latency against resource cost and implementation complexity. Ghorbian et al. (2024) survey the scheduling mechanisms underlying these behaviours and reach a similar conclusion about the absence of a dominant approach.

This trade-off is not uniform across an HRMS. A function serving a latency-sensitive interactive request, such as an employee checking a leave balance, is far more exposed to cold-start delay than a background function computing a monthly payroll run, and the two may reasonably warrant different design treatment. Section 4.3's response-time measurement is structured to distinguish these cases rather than reporting a single aggregate figure.

A further architectural evolution, and the one this project builds on directly, combines serverless execution with edge computing, deploying application code to data centres distributed geographically close to end users rather than to a single centralised region. Raith et al. (2023) describe this combination as addressing a specific weakness of conventional centralised serverless platforms: network latency to a distant region can dominate a

request's total response time regardless of how quickly the function executes, and moving execution to the edge reduces that network distance directly. Their review also identifies the open challenges the combination introduces, including maintaining state and data consistency across a geographically distributed set of execution locations, and the more constrained compute and memory budgets typical of lightweight edge runtimes.

Cloudflare Workers, the platform this project builds on, exemplifies the edge-serverless combination through a specific technical choice. Rather than executing each invocation inside a separate container or virtual machine, Workers runs application code inside lightweight V8 isolates, sharing an underlying process rather than requiring an operating-system boot per invocation (Cloudflare, 2026a). This is the mechanism behind the reduced cold-start latency the platform is generally reported to achieve relative to container-based serverless platforms. It should be stated plainly that this comparative performance claim derives from vendor and practitioner sources rather than from the peer-reviewed literature cited elsewhere in this chapter. The academic literature on serverless cold starts establishes the general problem and the mitigation landscape (Golec et al., 2024) but has not, to this project's knowledge, produced an independently verified benchmark of isolate-based platforms specifically. Comparative surveys of public Function-as-a-Service providers have begun to address this gap (Ekwe-Ekwe & Amos, 2024), but coverage of edge-isolate platforms remains thin. This is one point on which the present project's own response-time measurement (Section 4.3) can offer evidence, even at modest scale.

Taken together, this literature establishes serverless, edge-based computing as an architecture with real and well-documented benefits for scalability and cost, namely no idle capacity, automatic elasticity, and reduced network latency, alongside real and well-documented constraints around cold starts, distributed state management, and constrained per-invocation resources. What it has not established, and what Section 2.10 returns to, is how well the architecture holds up when applied to a system as data-heavy and workflow-heavy as a multi-tenant HRMS, rather than to the narrowly scoped compute tasks that dominate the existing literature.

2.6 Multi-Tenancy and Tenant Isolation in SaaS Platforms

Because the system under development serves several independent client organisations from one deployment, tenant isolation is a first-order design problem rather than an implementation detail. The question belongs to the multi-tenancy literature more broadly rather than to HR-specific research, since it is a general SaaS design problem rather than one unique to HR data.

Hattab and Belalem (2023), modelling multi-tenant service architectures using a multi-level Petri-net-based approach, characterise the central design tension as one between resource sharing, which pools infrastructure across tenants to reduce per-tenant cost, and isolation, which guarantees that one tenant's data, configuration, and load cannot affect another's. Their modelling formalises what is described more informally elsewhere as a spectrum

running from fully isolated, per-tenant infrastructure at one end, to fully shared infrastructure with tenant identity enforced only at the data layer at the other.

Three positions on this spectrum are conventionally distinguished. In the **silo** model, each tenant receives a dedicated database instance, giving the strongest isolation at the highest operational and financial cost, since schema migrations, backups, and monitoring must be applied per tenant. In the **bridge** model, tenants share a database instance but occupy separate schemas, giving intermediate isolation and permitting per-tenant schema customisation at the cost of more complex migration management. In the **pool** model, tenants share both database and schema, and isolation is enforced by a tenant discriminator column present on every tenant-owned row. The pool model is the cheapest and simplest to operate, and it permits query plan reuse across tenants, but it makes isolation a function of query discipline: a single query that omits its tenant filter becomes a cross-tenant data leak rather than merely a logic error.

{TABLE PLACEHOLDER — Table 2.2: A comparison table with six columns (Isolation model; Data separation mechanism; Isolation strength; Operational complexity; Per-tenant cost; Principal risk) and three rows (Silo — database per tenant; Bridge — schema per tenant; Pool — shared schema with tenant discriminator column). The Pool row should identify its principal risk as an unfiltered query producing cross-tenant leakage. Add a final column or footnote marking which model this project adopts. Table title goes ABOVE the table.}

Table 2.2: Multi-tenant data isolation models compared

The trade-off has a compounding effect on the cost problem identified in Section 1.2. Where a multi-tenant platform runs on shared provisioned infrastructure, that infrastructure must be sized for the combined peak of all tenants simultaneously. This produces a direct choice between over-provisioning, and so paying for capacity that most tenants use only rarely, and under-provisioning, and so risking one tenant's load degrading performance for every other tenant sharing that infrastructure. Hattab and Belalem's (2023) modelling formalises exactly this tension, but neither their work nor the HR-specific systems reviewed in Section 2.10 evaluates it against a serverless alternative in which per-request compute allocation could absorb tenant-level bursts individually rather than requiring the whole platform to be sized for every tenant's worst case at once.

Where the pool model is adopted, as it is in this project for the reasons set out in Section 3.5, the security literature is clear about the compensating controls it demands. Tenant context must be established from a verified credential rather than accepted from a client-supplied header, the tenant identifier must be applied at every resource query rather than only at the interface layer, and tenant ownership must be validated at the data-access layer rather than left to individual route authors (OWASP, 2025). These requirements shape the request pipeline and the shared query wrapper described in Sections 3.4 and 4.2.

Authentication mechanism is a related concern. Stateless token-based authentication using signed JSON Web Tokens has become the prevailing approach for single-page applications communicating with a stateless API, since it removes the need for server-side session storage, which is a meaningful advantage where the compute layer itself is stateless. Dalimunthe et al. (2023) evaluate JWT-based session management with HMAC signing in a

REST API context and report it suitable for cross-platform information systems, while noting that token revocation remains the principal weakness of the stateless approach, since a signed token remains valid until it expires. This informs the short token lifetime adopted in Section 3.7.

2.7 Applications of Cloud Technology in HR Systems

Having reviewed cloud, serverless, and multi-tenancy as general concerns, this section turns to how these technologies have been applied within HR systems specifically.

Cloud-based HR platforms have followed the broader shift toward digital HRM discussed in Section 2.2, moving from cloud as a hosting convenience toward cloud as the enabler of capabilities that would be impractical on-premise. Al-Mutawa and Al Mubarak (2025), studying cloud HRM adoption among small and medium-sized enterprises, found that these organisations value cloud-based systems specifically because they remove the need for upfront infrastructure investment while still providing access to recruitment, payroll, and performance functionality that would otherwise require several separate on-premise tools. Their study frames this as a scalability and flexibility benefit rather than merely a cost one: cloud HRM systems can expand with an organisation's headcount and functional needs without the step-change capital expenditure that on-premise systems demand at each growth threshold.

Maqueira Marín et al. (2022), in a systematic review of cloud computing in HRM, reach a broadly consistent conclusion while noting that the research agenda has been dominated by adoption and outcome studies. Their review calls for greater attention to the technical and architectural dimensions of these systems, which is precisely the dimension the present project addresses.

More recent industry-facing research has pushed toward explicitly multi-organisation, multi-tenant HR platforms. Ammupriya et al. (2025), presenting a cloud-based HR platform for scalable workforce management across multinational organisations, describe the specific demands such platforms place on the underlying architecture: the system must support organisational hierarchies that differ across client organisations, apply different compliance and policy rules per tenant, and scale independently for each tenant's usage pattern rather than treating the platform's overall load as a single undifferentiated pool. This confirms that the functional requirements this project targets, namely tenant-scoped organisational structure, independent per-tenant configuration, and tenant-level scalability, are recognised needs in current HR-platform research rather than a scope invented for this project alone.

Notably, none of the HR-specific studies reviewed in this section describe their platforms as built on a serverless architecture. The multi-tenant HR platforms in this literature are consistently framed as cloud-hosted in the general IaaS, PaaS, and SaaS sense reviewed in Section 2.4, with scalability achieved through conventional provisioned or container-orchestrated infrastructure rather than through request-scoped, serverless compute. The combination of multi-tenant HRM functionality delivered on a serverless, edge-native backend is the specific pairing the existing literature has not examined.

2.8 Payroll Automation and Statutory Compliance

Payroll is the module in which the consequences of manual processing are most acute, and it is treated here as a distinct theme because it carries obligations that other HR modules do not.

Zhao and Rabiei (2023) assess the feasibility of implementing a human resource payroll management system on cloud infrastructure and identify the drivers as reduced maintenance burden, elimination of local infrastructure cost, and improved accessibility. Their study frames payroll as a workload particularly well suited to cloud delivery because its computational demand is concentrated into predictable, periodic runs rather than distributed evenly across the month. This observation strengthens the serverless case made in Section 2.5: a workload that is intensely active for a short window and near-idle otherwise is precisely the shape for which per-execution billing outperforms continuous provisioning (Liu & Niu, 2023).

What the payroll literature addresses less thoroughly is statutory compliance as a system design problem rather than an accounting one. In the Nigerian context, an employer must operate progressive Pay-As-You-Earn income tax with a Consolidated Relief Allowance, remit mandatory pension contributions with distinct employee and employer components, deduct National Housing Fund contributions, and account for employer-borne levies to the Nigeria Social Insurance Trust Fund and the Industrial Training Fund. Each carries its own computation base, its own remittance deadline, and its own filing format. A payroll system that computes net pay correctly but leaves remittance tracking to a manual reminder process has automated only part of the obligation.

There is empirical support for treating this as a systems problem. Falana et al. (2024), examining digital tax administration and compliance in Nigeria, find that digitalising administration measurably improves compliance outcomes, with electronic processes reducing both the cost of compliance for the taxpayer and the cost of enforcement for the revenue authority. Read alongside the fragmentation problem set out in Section 2.3, this supports a design in which statutory rules are encoded into the computation path itself, deductions are derived from live system data rather than re-keyed figures, and remittance obligations are generated as tracked, dated tasks at the moment a payroll run is finalised. Section 4.2's payroll implementation follows this approach directly.

A further design consequence follows from auditability. Where a deduction is later disputed, whether by an employee or by a revenue authority, an aggregate deduction total is of limited evidential value. Storing each deduction category separately, with the figures resolved and persisted at computation time rather than recalculated on read, is what makes a payroll figure defensible after the fact. This principle informs the payroll schema presented in Section 3.5.

2.9 Artificial Intelligence and Large Language Models in HR Systems

The final substantive theme concerns the integration of artificial intelligence into HR systems, which has moved rapidly from predictive analytics toward conversational, generative interfaces.

Budhwar et al. (2023) set out both the opportunity and the hazard of generative artificial intelligence in HRM, arguing that these technologies will affect core HR functions substantially while raising unresolved questions about bias, transparency, accountability, and the appropriate division of labour between algorithmic and human judgement. Dasaklis et al. (2026), in a systematic review of the LLM-specific literature, map applications across recruitment and selection, training and development, and performance evaluation, and classify the field along both a technical axis, distinguishing discriminative from generative models, and a theoretical one covering conceptual modelling, empirical work, ethics and governance. Their review identifies governance and accountability as the least developed area, which is consistent with the picture in adjacent domains.

Recruitment has attracted the most sustained ethical scrutiny. Mori et al. (2025), reviewing 120 articles on artificial intelligence in recruitment and selection, organise the field around three ethical positions: a utilitarian view concerned with process efficiency, a justice view concerned with algorithmic bias and perceived fairness, and a rights view concerned with privacy, transparency, and data protection. Their review documents evidence that selection algorithms trained on unrepresentative data can produce systematically unfair outcomes for candidates in protected subgroups, and that candidates frequently prefer retaining human involvement in evaluation even where algorithmic assessment is more consistent.

This finding shapes a design decision in the present project. The recruitment module implemented here provides structured pipeline management, per-interviewer scorecards, and a complete audit timeline of hiring decisions, but it deliberately does not perform automated candidate ranking, résumé scoring, or algorithmic screening. Given the evidence on bias and on candidate perceptions of fairness, the design position adopted is that the system should improve the structure, consistency, and auditability of a human hiring decision rather than substitute for it.

The assistant module raises a different problem. A conversational interface over HR and payroll data must contend with the fact that a language model does not inherit the surrounding application's authorisation logic. The prevailing catalogue of risks for LLM applications ranks prompt injection as the leading vulnerability class, in which crafted input causes a model to disregard its instructions, and separately identifies excessive agency, in which a model is granted more functionality, permission, or autonomy than its task requires, as a distinct risk arising from the combination of excessive functionality, excessive permissions, and excessive autonomy (OWASP, 2025). Where a model is given tools that read employee records, an injection that succeeds becomes a data-access event rather than merely an incorrect answer.

The mitigation implied by this literature is architectural rather than prompt-based. If the model's arguments cannot be trusted, then authorisation must be re-derived inside each tool implementation from the caller's verified identity, the tool set must be minimal and read-only, the agent loop must be bounded, and every invocation must be logged. Section 3.7 sets out this design and Section 4.2 reports its implementation. The literature reviewed here

establishes both that such assistants are being adopted in HR contexts and that the governance patterns for doing so safely are not yet settled, which is the basis on which this project treats its assistant module as a contribution rather than a feature.

2.10 Review of Existing HR Management Systems

This section reviews existing systems directly, covering both those described in the academic literature and the commercial platforms in general use, and assesses them against the feature set and architecture this project adopts.

2.10.1 Systems described in the academic literature

Academic contributions in this space fall broadly into two groups. The first comprises design-and-implementation papers presenting a working system for one or a small number of HR functions. These typically cover employee records with attendance, leave, or payroll, implemented on a conventional three-tier web stack with a relational database on a provisioned server. Their contribution is usually a demonstration that the target processes can be digitised successfully in a specific organisational setting, and they commonly report improvements in processing time and error rate over the manual baseline they replace. Their limitation, for the purposes of the present project, is scope and architecture: they address a subset of the employee lifecycle, they are single-tenant, and the deployment model is assumed rather than examined.

The second group comprises platform-level and review contributions. Zhao and Rabiei (2023) evaluate cloud-based payroll implementation feasibility, establishing the case for cloud delivery of payroll specifically but treating cloud as a hosting decision rather than an architectural one. Ammupriya et al. (2025) present a multi-tenant cloud HR platform for multinational workforce management and address the tenant-differentiated hierarchy and compliance requirements directly, but on conventional provisioned infrastructure. Destriani et al. (2024) review HRMS implementation challenges across the literature and identify integration difficulty, cost, and limited technical capability as the recurring obstacles, treating infrastructure overhead as a fact of deployment rather than as something a different architecture might reduce. Maqueira Marín et al. (2022) survey cloud computing in HRM at the field level and explicitly note the shortage of architectural research relative to adoption research.

2.10.2 Commercial platforms

The commercial market includes established enterprise suites, mid-market platforms, and regionally focused products serving specific statutory environments. Assessed at the level of publicly documented capability, several characteristics are common across them.

They are functionally comprehensive, typically covering workforce records, attendance, leave, payroll, performance, and recruitment within a single product, which confirms that the functional scope this project targets is well preceded rather than unusual. They are delivered as multi-tenant SaaS, with tenant separation and configurability treated as a product requirement. Where they serve a specific national statutory environment, they

encode that jurisdiction's payroll rules, which is the same requirement this project addresses for the Nigerian context.

Three limitations are equally consistent. First, the underlying architecture is not publicly documented in a form that permits academic comparison; scalability is asserted as a product property rather than described as an architectural one. Second, where deployment architecture is disclosed at all, it is described in conventional provisioned or container-orchestrated terms rather than as request-scoped serverless compute. Third, pricing is structured per employee per month rather than per unit of computation, which means the cost model presented to the customer is decoupled from the cost structure of the underlying infrastructure, and neither model is the usage-based structure this project evaluates.

{TABLE PLACEHOLDER — Table 2.3: A comparative matrix. Rows: the systems reviewed in this section, grouped as (a) academic single-function systems, (b) Zhao & Rabiei (2023) cloud payroll, (c) Ammupriya et al. (2025) multi-tenant cloud HR platform, (d) representative commercial HRMS platforms, and (e) ZenHR (this project). Columns: Full employee lifecycle coverage; Multi-tenant; Statutory payroll engine; Serverless/edge-native backend; Usage-based cost model; Integrated AI assistant with scoped authorisation; Public evaluation of architecture. Use a tick, cross, or partial marker in each cell. The final row should be the only one carrying ticks across the full set. Table title goes ABOVE the table.}

Table 2.3: Comparative review of existing HR management systems against this project

2.10.3 What the comparison shows

No system reviewed here, academic or commercial, combines full-lifecycle multi-tenant HRMS functionality with a serverless, edge-native backend, a jurisdiction-specific statutory payroll engine, and an authorisation-scoped language-model assistant, and none publishes an evaluation of that architecture. Each element exists somewhere in the literature or the market. The combination, and the evidence about how it performs, does not.

2.11 Limitations of Existing Systems

This section consolidates the limitations surfaced across Sections 2.2 through 2.10 into a direct statement of what existing systems do not adequately address and to which this project's design responds.

Fixed-capacity cost structure. Conventional cloud HRIS deployments, including every commercial platform characterised in Section 2.10, run on provisioned virtual machines or containers that must be sized for peak demand and paid for regardless of actual usage. Mkhize et al. (2025) confirm that cost management remains a live concern even among organisations that have already adopted cloud services, rather than a problem that cloud adoption alone resolves. The scalability claims made for these systems describe the platform's capacity to onboard more tenants onto existing provisioned infrastructure, not a departure from provisioned infrastructure itself.

Compounding cost and performance risk under multi-tenancy. Where existing systems are multi-tenant, the fixed-capacity limitation compounds rather than merely repeats, since shared provisioned infrastructure must be sized for the combined peak of all tenants at once. Hattab and Belalem (2023) formalise the underlying resource-sharing-versus-isolation tension, but neither their work nor the HR-specific systems reviewed evaluates it against a serverless alternative capable of absorbing tenant-level bursts individually.

Technical and maintenance overhead treated as unavoidable. Destriani et al. (2024) identify limited in-house technical capability as a recurring practical obstacle to HRMS implementation and maintenance. Systems that require an organisation to size instances, patch servers, and manage scaling configuration place that burden on precisely the smaller, resource-constrained organisations least equipped to carry it, and the existing literature treats this as a cost of deploying cloud infrastructure rather than as something a different architectural choice could reduce.

Narrow treatment of serverless architecture outside general-purpose compute. Section 2.5 established that serverless and serverless-edge computing is well studied as a general architecture, with documented cold-start trade-offs (Golec et al., 2024; Ghorbian et al., 2024) and documented latency benefits from edge deployment (Raith et al., 2023). That literature is nonetheless dominated by infrastructure-level use cases. None of it, and none of the HR-specific systems reviewed in Section 2.10, evaluates serverless architecture against a system with an HRMS's particular operational profile: structured relational data, unstructured file storage, multi-step cross-departmental workflows, computationally non-trivial statutory payroll, and strict tenant isolation, all in one deployment.

Statutory compliance treated as computation rather than as tracked obligation. The payroll literature addresses correct computation but gives little attention to remittance as a system-enforced process. Where compliance deadlines are left to manual tracking, the system has automated the arithmetic while leaving the obligation that carries the regulatory penalty unmanaged, despite evidence that digitalising administration improves compliance outcomes (Falana et al., 2024).

Unsettled governance for language models over sensitive HR data. Assistants are being introduced into HR platforms while the governance patterns remain unresolved (Budhwar et al., 2023; Dasaklis et al., 2026), and the dominant catalogued risks for such systems, prompt injection and excessive agency, concern exactly the boundary between the model and the data it can reach (OWASP, 2025). No system reviewed here publishes an architectural treatment of how an assistant's data access is bounded relative to the surrounding application's existing authorisation model.

{FIGURE PLACEHOLDER — Figure 2.2: A three-circle Venn diagram showing the intersection of the three literature streams reviewed in this chapter. Circle A: "Multi-tenant HRMS functionality" (populated by Ammupriya et al., 2025; commercial platforms). Circle B: "Serverless, edge-native architecture" (populated by Wen et al., 2023; Raith et al., 2023; Golec et al., 2024). Circle C: "Statutory payroll automation and governed AI access" (populated by Zhao & Rabiei, 2023; Falana et al., 2024; OWASP, 2025). Label the pairwise overlaps as partially addressed, and label the central three-way intersection as the research

gap this project occupies, with the label "ZenHR — this project". Caption goes BELOW the figure.}

Figure 2.2: Intersection of literature streams and the research gap addressed by this project

2.12 Summary of Literature Review

This chapter has traced a path from the fundamentals of human resource management through to the specific architectural gap this project addresses.

Section 2.2 established HRMS as a well-studied concept at the level of what such systems should do, tracing the development of the field toward digital HRM (Vu, 2024) and noting that the literature has increasingly framed such systems as active shapers of HR practice rather than passive record stores. Section 2.3 showed that the traditional and fragmented approaches this evolution moved away from suffered from data fragmentation, manual error-prone processing, fixed on-premise infrastructure, and weak cross-functional workflow support, the last of which this project's transition modules respond to directly (Destriani et al., 2024). Section 2.4 introduced cloud computing as the general-purpose response to the infrastructure dimension of that problem, establishing its documented benefits and adoption barriers as the baseline against which this project's architecture is measured (Mkhize et al., 2025).

Section 2.5 traced the further step from conventional cloud provisioning to serverless, edge-native computing, establishing its benefits of eliminated idle capacity, automatic elasticity, and reduced network latency, alongside its documented trade-offs around cold-start latency, scheduling, and distributed state (Ghorbian et al., 2024; Golec et al., 2024; Raith et al., 2023; Shafiei et al., 2022; Wen et al., 2023), and noting that the cost advantage is workload-dependent in a way that favours the bursty profile of HR activity (Liu & Niu, 2023). Section 2.6 reviewed multi-tenancy as a distinct design problem, positioning the silo, bridge, and pool isolation models along the resource-sharing-versus-isolation spectrum formalised by Hattab and Belalem (2023) and setting out the compensating controls the pool model demands (Dalimunthe et al., 2023; OWASP, 2025).

Section 2.7 returned the general architecture to the HR domain, establishing that current HR-specific research addresses multi-tenancy and scalability needs closely resembling this project's own but does so exclusively on conventional provisioned infrastructure (Al-Mutawa & Al Mubarak, 2025; Ammupriya et al., 2025; Maqueira Marín et al., 2022). Section 2.8 examined payroll automation and statutory compliance, establishing both the suitability of payroll's periodic workload for cloud and serverless delivery (Zhao & Rabiei, 2023) and the evidence that digitalising administration improves compliance outcomes in the Nigerian context (Falana et al., 2024). Section 2.9 reviewed the emerging literature on artificial intelligence in HR, establishing the ethical constraints that shape this project's deliberate exclusion of algorithmic candidate ranking (Mori et al., 2025) and the security constraints that shape its assistant's tool-scoped design (Budhwar et al., 2023; Dasaklis et al., 2026; OWASP, 2025).

Section 2.10 reviewed existing HR management systems directly, academic and commercial, and Section 2.11 consolidated the findings into six limitations: fixed-capacity cost structure, compounding multi-tenant cost and performance risk, technical overhead treated as unavoidable rather than architecturally addressable, a serverless literature dominated by infrastructure-level rather than business-application use cases, statutory compliance treated as computation rather than as tracked obligation, and unsettled governance for language models operating over sensitive HR data.

Taken together, this review establishes a clear and specific gap. The HR feature set this project implements is well understood and well preceded. The serverless edge architecture it builds on is well studied as a general-purpose computing model. The statutory payroll rules it encodes are documented in law and in accounting practice. The risks of exposing a language model to sensitive data are catalogued. What has not been evaluated in either the academic or the commercial literature reviewed here is the combination: a full-lifecycle, multi-tenant HRMS with an embedded statutory payroll engine and an authorisation-scoped AI assistant, built entirely on serverless, edge-native infrastructure, and assessed on both performance and cost. This is precisely the gap the aim and objectives set out in Section 1.3 respond to, and which Chapter Three begins to address through the system's architecture and design.

CHAPTER THREE: SYSTEM METHODOLOGY AND DESIGN

3.1 Introduction

This chapter translates the methodology introduced in Section 1.5 into the concrete architecture, requirements, and design decisions that underpin the system's implementation. It begins with the physical system architecture and the platform services it is composed of (Section 3.2), then presents the conceptual framework describing how data and control move through the system independently of the platform implementing it (Section 3.3). Section 3.4 sets out the functional and non-functional requirements the system must satisfy, each traced back to an objective in Section 1.3. Section 3.5 presents the system design through use-case, data-flow, entity-relationship, and request-pipeline diagrams. Section 3.6 details the database design on Cloudflare D1, including the tenant-isolation decision and the schemas of the modules carrying the most design weight. Section 3.7 covers the interface design, and Section 3.8 consolidates the security design, including an explicit account of one known deviation from the intended authentication model.

Each design decision in this chapter is justified against the objectives in Section 1.3 and the gap established in Chapter Two, rather than presented as an unmotivated technical choice. Where a decision involved a trade-off, the trade-off is stated together with the reason the chosen side was preferred.

3.2 System Architecture

The system is implemented as a two-tier client-server application deployed entirely on Cloudflare's serverless platform. The two tiers are independently deployable and communicate exclusively over HTTPS using JSON. There is no server-side rendering and no shared runtime between them.

3.2.1 Platform components

Cloudflare Pages (presentation tier). The frontend is a React single-page application compiled by Vite and deployed as static assets. It holds no direct connection to the database or object storage; every data access is mediated by the API tier.

Cloudflare Workers (application and API tier). The backend is a stateless HTTP API implemented with the Hono web framework and deployed as a Worker. Rather than running on a provisioned server or container, each incoming request is handled by a lightweight V8 isolate distributed across Cloudflare's global network rather than bound to a single centralised region (Cloudflare, 2026a; Raith et al., 2023). The Worker hosts all authentication, tenant resolution, authorisation, and business logic for every functional module.

Cloudflare D1 (relational data tier). D1 is a serverless SQL database built on SQLite (Cloudflare, 2026b). It stores all structured data: company records and settings, organisational structure, employee records, attendance and leave, recruitment pipeline, payroll configuration and results, performance records, benefits, and audit history. The schema is defined declaratively in TypeScript through Drizzle ORM and realised as sequential SQL migrations.

Cloudflare R2 (object storage tier). R2 provides S3-compatible object storage for unstructured files: employee documents, candidate résumés, and generated payroll artefacts (Cloudflare, 2026c). Objects are referenced from corresponding rows in D1 rather than duplicated into the relational database, so that structured metadata and file content remain queryable together while the file itself lives in storage designed for it.

Cloudflare Workers AI (inference tier). Workers AI provides hosted large-language-model inference for the assistant module, invoked from the same Worker that serves the rest of the API and therefore subject to the same authentication and tenant-resolution pipeline.

{FIGURE PLACEHOLDER — Figure 3.1: A high-level system architecture diagram. At the top, a browser client. Below it, Cloudflare Pages hosting the React SPA. An arrow labelled "HTTPS / JSON" runs from Pages down to a box labelled "Cloudflare Worker — zenhr-api (Hono)", annotated with "JWT authentication, tenant resolution, RBAC, business logic". From the Worker, three arrows fan out to: "Cloudflare D1 — relational data", "Cloudflare R2 — object storage", and "Cloudflare Workers AI — LLM inference". Show clearly that no arrow runs from the client directly to D1, R2, or Workers AI. Caption goes BELOW the figure.}

Figure 3.1: High-level system architecture

3.2.2 Backend layered architecture

The Worker codebase follows a strict layered separation of concerns, adapted from conventional model-view-controller structure for an API-only backend. A request passes downward through the layers and a response passes back up the same path.

Routes define the mapping from HTTP path and method to a middleware chain and a controller function. They contain no logic.

Middlewares perform authentication, tenant resolution, and authorisation before any handler executes.

Controllers parse the request, invoke exactly one service method, and shape the HTTP response. They hold no business logic.

Services contain all business logic, computation, and orchestration. The payroll engine, leave rules, transition state derivation, and assistant tool implementations all live here.

Models are the Drizzle table definitions. This is the only layer that knows the shape of the database.

This arrangement matters for more than tidiness. Because business logic is confined to the service layer, the payroll computation can be unit-tested against a mocked database without standing up any infrastructure, and because table shape is confined to the model layer, a schema change surfaces as a compile-time type error in every query built on it rather than as a runtime failure in production.

One documented deviation exists. The Settings and Control Centre routes invoke their service classes directly, with no intervening controller. This was a deliberate choice made because those routes perform simple configuration reads and writes with no request shaping worth a separate layer, and it is recorded here as an acknowledged inconsistency rather than an oversight.

{FIGURE PLACEHOLDER — Figure 3.2: A vertical layered architecture diagram of the backend. From top to bottom, six stacked bands with a downward arrow through them: (1) HTTP request; (2) routes — path to middleware chain to controller; (3) middlewares — authMiddleware (JWT), requireRole / requirePermission, tenantMiddleware; (4) controllers — parse request, call one service, shape response; (5) services — all business logic and computation; (6) models (Drizzle) — typed table schemas; and at the base, Cloudflare D1. Annotate the middleware band with "rejection here short-circuits the request". Caption goes BELOW the figure.}

Figure 3.2: Layered architecture of the Workers backend

3.2.3 Architectural rationale

This architecture is a direct implementation of the serverless, edge-native model reviewed in Section 2.5. Three consequences follow, each addressing a problem identified in Section 1.2.

Compute is allocated per request. There is no provisioned instance to size for peak demand, and no idle instance to pay for between payroll runs. This addresses the fixed-capacity cost structure identified in Section 2.11 directly, and it addresses it in a way that is particularly favourable for payroll's workload shape, which is intensely active for a short window each month and near-idle otherwise (Liu & Niu, 2023).

The data and storage tiers scale independently of the compute tier and of each other. A recruitment surge that increases request volume does not require additional database capacity to have been provisioned in advance, and document upload volume does not compete with query throughput.

Operational overhead is transferred to the platform. There are no operating systems to patch, no instances to size, and no autoscaling rules to configure, which addresses the technical-capability obstacle that Destriani et al. (2024) identify as a recurring barrier to HRMS implementation and maintenance in resource-constrained organisations.

The trade-offs are equally real and are treated as design constraints rather than dismissed. Cold-start latency affects interactive requests more than batch computation (Golec et al., 2024), which is why the response-time evaluation in Chapter Four distinguishes between the two categories. Edge execution constrains per-invocation compute and memory relative to

centralised serverless platforms (Raith et al., 2023), which is why the payroll engine computes per employee per invocation rather than attempting a whole-company run in a single execution.

3.3 Conceptual Framework of the Proposed System

Where Section 3.2 described the physical architecture, this section presents the logical model of how data and control flow through the system to satisfy the objectives in Section 1.3, independent of the platform implementing it.

The system is conceptually organised into three layers.

The tenant and access layer governs who is interacting with the system and on whose behalf. Every request is first resolved to a specific company and a specific role within that company before anything else happens. This layer is the conceptual home of the multi-tenant isolation strategy and the role-based access control set out in Section 3.8. No request reaches the layers below without passing through this resolution step, and a failure at this step terminates the request.

The HR process layer contains the functional modules identified in Section 1.4. Conceptually, each module is a set of state transitions applied to employee-related records: a transition case advancing as its constituent tasks complete, a leave request moving from pending to approved or rejected, a candidate progressing through the recruitment pipeline, a payroll run moving from preview through submission and approval to paid. This is the layer where the objectives are fulfilled, and it is the layer whose behaviour Chapter Four reports as implemented and tested.

The data persistence layer durably records the outcomes of those transitions, split between structured state held in D1 and unstructured artefacts held in R2. This layer is deliberately passive. Validation, sequencing, and business rules live in the process layer, so that the schema remains a straightforward reflection of system state rather than an implicit second copy of the business logic. This separation is what makes the payroll engine testable in isolation and what keeps a schema migration from silently changing a business rule.

Every request follows the same conceptual path: it enters the tenant and access layer, is resolved to a company and role, is handled by the relevant process module, and results in a read or write against the persistence layer, with the response flowing back up the same three layers. This uniformity is what makes the serverless implementation tractable. Because every request follows an identical shape regardless of which module it touches, each module can be implemented as an independent set of routes without shared stateful infrastructure between them. That property is precisely what makes per-request scaling a good fit for this system rather than a technology choice imposed on top of it.

{FIGURE PLACEHOLDER — Figure 3.3: A three-layer conceptual framework diagram. Top layer, "Tenant and access layer", listing: identity verification, company resolution, role resolution, permission narrowing. Middle layer, "HR process layer", showing the functional modules as boxes: workforce and organisation; transitions (onboarding/offboarding); attendance and overtime; leave; recruitment and ATS; payroll and statutory compliance;

performance; benefits and wellness; assets, learning, surveys and support; control centre and audit; AI assistant. Bottom layer, "Data persistence layer", split into "Structured state — D1" and "Unstructured artefacts — R2". A single arrow should run down through all three layers and back up, labelled "every request follows this path". Caption goes BELOW the figure.}

Figure 3.3: Conceptual framework of the proposed system

3.4 System Requirements

This section translates the conceptual framework into specific, verifiable requirements. Functional requirements state what the system must do and are grouped by module, each tracing back to an objective in Section 1.3. Non-functional requirements state the qualities the system must exhibit while doing it.

3.4.1 Functional Requirements

Tenancy, identity, and access control (Objectives 2 and 3)

- **FR1:** The system shall allow a new company to be registered as a tenant with its own isolated data context, configuration, and audit history.
- **FR2:** The system shall authenticate users by email and password, storing credentials only as salted hashes, and shall issue a signed token on successful authentication carrying the user's identity, company, and role.
- **FR3:** The system shall support a temporary password reset that forces the user to set a new password at next login.
- **FR4:** The system shall enforce role-based access control across six fixed roles: Super Administrator, HR Administrator, Manager, Employee, Payroll Officer, and Recruiter.
- **FR5:** The system shall support optional custom roles carrying a permission matrix over defined modules and actions, which may narrow but never widen the access granted by a user's fixed role.
- **FR6:** The system shall scope every query touching company-owned data to the authenticated user's company, such that no data belonging to one company is retrievable through a session authenticated against another.

Workforce and organisational management (Objective 4)

- **FR7:** The system shall support creation, retrieval, update, and deactivation of employee records, including employment details, statutory identifiers, and bank details.
- **FR8:** The system shall support definition of a company's organisational structure through departments and locations, including nested department hierarchy.
- **FR9:** The system shall represent reporting lines through a manager reference on the employee record, and shall use that reference to scope manager-facing views and approvals.
- **FR10:** The system shall maintain emergency contacts and a document store per employee.

- **FR11:** The system shall expose a per-employee audit history showing changes made to that employee's record.

Onboarding and offboarding transitions (Objective 4)

- **FR12:** The system shall model onboarding and offboarding as transition cases composed of task checklists categorised by responsible function.
- **FR13:** The system shall derive a transition's overall stage from the completion state of its constituent tasks rather than allowing stage to be set independently, so that recorded status cannot diverge from actual progress.
- **FR14:** The system shall record, for an offboarding case, the separation reason, the handover recipient, and whether an exit interview is scheduled.
- **FR15:** The system shall route an employee whose status is onboarding into a dedicated self-service onboarding flow at first login rather than the standard portal.

Attendance and overtime (Objective 4)

- **FR16:** The system shall record employee clock-in and clock-out events, permitting multiple sessions within a single day rather than overwriting a prior entry.
- **FR17:** The system shall optionally capture geolocation coordinates at clock-in and clock-out.
- **FR18:** The system shall compute worked hours and overtime hours per attendance record.
- **FR19:** The system shall classify attendance against company-configured working hours, grace period, and working-day calendar.
- **FR20:** The system shall support an overtime request workflow in which an employee submits hours worked outside a scheduled shift with a justification and a manager approves or rejects it.

Leave management (Objective 4)

- **FR21:** The system shall maintain leave entitlement per employee per leave type.
- **FR22:** The system shall allow an employee to submit a leave request with a date range, reason, and optional supporting attachment.
- **FR23:** The system shall compute days taken from approved requests at query time rather than maintaining a separately mutated running balance, so that entitlement and consumption cannot drift apart.
- **FR24:** The system shall restrict a manager's visible leave requests to their own direct reports, while allowing HR and Super Administrators company-wide visibility.

Recruitment and applicant tracking (Objective 6)

- **FR25:** The system shall support job requisitions with an approval workflow covering requested, approved, open, on hold, filled, cancelled, and rejected states.
- **FR26:** The system shall record the requester, the reviewer, the budget range, and the business justification for each requisition.
- **FR27:** The system shall maintain a candidate pipeline progressing through applied, screening, interview, offer, and hired or rejected states.

- **FR28:** The system shall support scheduling of interviews and capture of per-interviewer scorecards, so that a panel records individual assessments rather than a single shared verdict.
- **FR29:** The system shall record offers made to candidates and their outcomes.
- **FR30:** The system shall append every pipeline transition, interview, and offer event to a per-candidate timeline, providing a complete record of the hiring decision.
- **FR31:** The system shall expose a public, unauthenticated careers page listing publicly flagged open requisitions and accepting external applications.

Payroll and statutory compliance (Objective 5)

- **FR32:** The system shall maintain per-company payroll configuration covering pay cycle, cut-off date, payment day, and the applicable statutory rates.
- **FR33:** The system shall maintain an editable catalogue of salary components and pay grades.
- **FR34:** The system shall prorate gross pay for employees hired part-way through a pay period, scaling by the proportion of the period worked.
- **FR35:** The system shall split gross pay into basic salary and allowances according to the configured component definitions.
- **FR36:** The system shall compute employee pension contributions from the pensionable base and separately compute and report the employer contribution without deducting it from the employee.
- **FR37:** The system shall compute National Housing Fund deductions from basic salary.
- **FR38:** The system shall compute Nigeria Social Insurance Trust Fund and Industrial Training Fund levies as employer-borne costs, reporting them for remittance while excluding them from the employee's net pay.
- **FR39:** The system shall compute Pay-As-You-Earn income tax on an annualised basis, applying the Consolidated Relief Allowance and pension relief before running the resulting taxable income through the company's configurable progressive tax bands, and dividing the annual result across the pay periods in the year.
- **FR40:** The system shall deduct active loan repayments at the lesser of the scheduled instalment or the outstanding balance, and shall close a loan once fully repaid.
- **FR41:** The system shall support a payroll run lifecycle of preview, submission, approval or rejection, and marking as paid; shall persist a payslip per employee per run; shall generate statutory remittance obligations with their due dates when a run is marked paid; and shall produce bank disbursement and scheme-specific remittance export files.
- **FR42:** The system shall surface a pre-run exception list flagging employees missing bank details, salary configuration, or statutory identifiers before a run is submitted.

Performance management (Objective 6)

- **FR43:** The system shall support company-wide review cycles composed of named stages, each with its own start and due dates.
- **FR44:** The system shall support employee self-assessments covering achievements, challenges, skill ratings, and development goals, subsequently rated and commented on by the employee's manager.

- **FR45:** The system shall support peer reviews requiring nomination and manager approval before submission, and upward reviews that do not require nomination.
- **FR46:** The system shall support goals scoped to an individual, team, department, or the company, with progress tracking and optional linkage to a parent goal for alignment.
- **FR47:** The system shall support lightweight peer recognition entries.
- **FR48:** The system shall record assigned, in-progress, and completed training per employee.

Benefits, wellness, learning, assets, surveys, and support (Objective 6)

- **FR49:** The system shall maintain a catalogue of benefit plans with employer and employee cost split and coverage limits, and shall support employee enrolment and dependent registration.
- **FR50:** The system shall support wellness programmes with per-participant progress tracking.
- **FR51:** The system shall support a claims workflow covering submission, review, and approval or rejection, posting approved wellness claims against the employee's wellness budget.
- **FR52:** The system shall maintain a company asset registry recording category, serial number, condition, purchase value, and assignment status, with an employee-facing view of their own assigned assets.
- **FR53:** The system shall support course creation, assignment to employees, and progress tracking through assigned, in progress, and completed states.
- **FR54:** The system shall support administrator-authored surveys targeted company-wide or by department, composed of ordered rating, free-text, and multiple-choice questions, with optionally anonymous responses.
- **FR55:** The system shall support an internal helpdesk with categorised, prioritised tickets and a threaded message log per ticket.

Control centre and audit (Objectives 3 and 6)

- **FR56:** The system shall support company profile, security settings, and operational calendar configuration including working days and public holidays.
- **FR57:** The system shall support custom role and permission management and API key issuance.
- **FR58:** The system shall support transactional email template management and a third-party integration catalogue.
- **FR59:** The system shall support a full export of a company's data across every table belonging to that tenant.
- **FR60:** The system shall write an audit entry for sensitive administrative actions, recording the actor, action, module, severity, descriptive detail, and originating IP address, and shall make that log searchable, filterable, and exportable.

AI assistant (Objective 7)

- **FR61:** The system shall expose a conversational endpoint accepting a natural-language question from an authenticated user.

- **FR62:** The system shall provide the language model with a fixed set of read-only tools rather than direct database access, and shall not expose any tool capable of creating, modifying, or deleting a record.
- **FR63:** Each tool implementation shall independently re-derive the caller's authorisation from the verified session rather than trusting arguments supplied by the model, and shall return a structured refusal rather than data where the caller is not entitled to the requested information.
- **FR64:** The system shall bound the agent loop to a fixed maximum number of tool-calling iterations, returning a request to narrow the question when the bound is reached.
- **FR65:** The system shall reject any request to the assistant endpoint that does not present a fully verified token carrying a role claim.
- **FR66:** The system shall log every assistant query with the company, employee, role, question, and tools invoked.

3.4.2 Non-Functional Requirements

Scalability and performance

- **NFR1:** The compute tier shall scale per request without manual provisioning of additional capacity.
- **NFR2:** Interactive read operations shall complete within a response time acceptable for a self-service HR tool, measured empirically in Chapter Four rather than assumed.
- **NFR3:** The data and object storage tiers shall scale independently of the compute tier and of each other, so that one company's usage does not require capacity to be provisioned on behalf of every other company.
- **NFR4:** Batch operations such as a payroll run shall be structured so that per-invocation compute stays within the platform's execution limits.

Multi-tenancy and isolation

- **NFR5:** No company's data shall be retrievable, directly or indirectly, through a request authenticated against a different company.
- **NFR6:** Tenant scoping shall be enforced at the data-access layer rather than left to individual query authors, so that the guarantee is structural rather than dependent on developer recall at each call site.
- **NFR7:** A company's configuration shall be modifiable independently of every other company's configuration.

Security

- **NFR8:** All traffic between the client, the Worker, and the platform data services shall be encrypted in transit.
- **NFR9:** Passwords shall be stored only as salted hashes and never in recoverable form.
- **NFR10:** Access to any record or action shall follow the principle of least privilege, granting only what the user's role requires.

- **NFR11:** Authorisation shall be re-validated server-side on every request and shall never rely on the frontend having hidden a control.
- **NFR12:** Audit entries shall be written by an insert-only path, with no route exposing update or deletion of an audit record.

Cost efficiency

- **NFR13:** The system shall incur no compute cost during periods of zero request activity.

Reliability and maintainability

- **NFR14:** A fault in one module's routes shall not prevent other modules from continuing to serve requests.
- **NFR15:** Business logic shall be testable in isolation from the database and from the HTTP layer.
- **NFR16:** A new company shall be onboardable without a code change or redeployment.
- **NFR17:** Schema changes shall be applied through versioned, sequential migrations rather than ad-hoc alteration.

Usability

- **NFR18:** The interface shall present only the functions and data relevant to the authenticated user's role and company.

3.5 System Design

3.5.1 Use Case Diagram

Six actor roles interact with the system, consistent with FR4. Role is a fixed attribute of the employee record; a user becomes a manager in practice by having other employees' manager reference point at them, rather than through a separate hierarchy structure.

Table 3.1: System actors and their principal capabilities

Role	Scope	Principal capabilities
Super Administrator	Company-wide, highest privilege	All HR Administrator capabilities, plus company settings, custom role management, and API key issuance

HR Administrator	Company-wide	Workforce records, organisational structure, transitions, attendance and leave administration, payroll configuration and run approval, performance cycles, benefits administration, reporting, control centre
Payroll Officer	Payroll domain	Payroll dashboard, run preview and submission, payroll reporting, exception review
Recruiter	Recruitment domain	Requisitions, candidate pipeline, interview scheduling, scorecards, offers, recruitment analytics
Manager	Direct reports only	Team attendance and leave approval, overtime approval, team goal assignment, self-assessment review, peer-review nomination approval, team reporting
Employee	Own record only	Clock in and out, leave application, payslip access, profile maintenance, benefit enrolment, course and survey completion, self-assessment submission, support tickets

{FIGURE PLACEHOLDER — Figure 3.4: A use-case diagram. Six actor stick figures on the left and right margins: Super Administrator, HR Administrator, Payroll Officer, Recruiter, Manager, Employee. A large system boundary rectangle labelled "ZenHR" in the centre containing grouped use-case ovals: "Manage workforce and organisation", "Run onboarding and offboarding", "Record and approve attendance", "Apply for and approve leave", "Manage recruitment pipeline", "Configure and run payroll", "Conduct performance reviews", "Administer benefits and wellness", "Manage assets, learning and surveys", "Raise and resolve support tickets", "Configure settings and review audit log", "Query the AI assistant". Inside the boundary and above the use cases, place a horizontal band labelled "Tenant and role resolution — every request passes through this first". Draw association lines from each actor only to the use cases their role permits. Caption goes BELOW the figure.}

Figure 3.4: Use case diagram of the ZenHR system

3.5.2 Data Flow Diagram

A data flow diagram shows how data moves between external entities, system processes, and data stores, without describing control logic or timing. This section presents a

context-level diagram treating the system as a single process, followed by a decomposition of one representative module.

At context level, the external entities are the six actor roles together with two parties outside the system boundary: external job applicants, who interact only with the public careers page, and the hosted inference service, which receives a prompt and tool definitions and returns either a tool-call instruction or a natural-language response. Data flows inward as configuration, requests, attendance events, and applications, and outward as records, payslips, export files, notifications, and reports.

{FIGURE PLACEHOLDER — Figure 3.5: A context-level (Level 0) data flow diagram. A single central circle labelled "ZenHR system". External entity rectangles arranged around it: HR Administrator (flows: configuration, approvals in; reports, records out); Manager (flows: approvals, scores in; team records out); Employee (flows: clock events, leave requests, self-assessments in; payslips, balances out); Payroll Officer (flows: run submission in; payroll reports and export files out); Recruiter (flows: requisitions, scorecards in; pipeline analytics out); External applicant (flows: application in; acknowledgement out); Hosted inference service (flows: prompt and tool definitions out; tool call or answer in). Caption goes BELOW the figure.}

Figure 3.5: Context-level data flow diagram

Decomposing the payroll module shows the pattern the other modules follow, and is chosen here because payroll draws on more data stores than any other process. A payroll run begins with the operator selecting a period; the system reads employee records, salary configuration, statutory rates, tax bands, and active loans; computes a payslip per employee; and returns the result for preview without persisting it. On submission, the computed payslips and the run header are written. On approval and marking as paid, three further effects follow within the same operation: loan balances are decremented and repayment records created, statutory remittance obligations are generated with their due dates attached, and the export files are made available.

{FIGURE PLACEHOLDER — Figure 3.6: A Level 1 data flow diagram decomposing the payroll process. Processes as circles: 1.0 "Validate run inputs and flag exceptions"; 2.0 "Compute payslip per employee"; 3.0 "Persist run and payslips"; 4.0 "Apply loan repayments"; 5.0 "Generate remittance obligations"; 6.0 "Produce export files". Data stores as open-ended rectangles: D1 employees; D1 payroll_settings; D1 tax_brackets; D1 salary_components; D1 loans; D1 payroll_runs; D1 payslips; D1 loan_repayments; D1 compliance_tasks. External entity: Payroll Officer. Show the preview path (1.0 → 2.0 → back to Payroll Officer) as distinct from the persistence path (2.0 → 3.0 → 4.0, 5.0, 6.0). Caption goes BELOW the figure.}

Figure 3.6: Level 1 data flow diagram for the payroll process

3.5.3 Entity Relationship Diagram

The database comprises approximately fifty tables across twelve functional domains. A diagram showing every table at readable size is not practical, so the entity-relationship

diagram presented here shows the core entities and the relationships that carry structural weight, with the remaining tables following the same scoping pattern.

Two structural conventions hold throughout. First, nearly every table carries a company reference, either directly or transitively through its relationship to the employee record; this is what makes tenant scoping enforceable at query level. Second, where a table stores a file, it stores an object key referencing R2 rather than the file content itself, so that no binary data is duplicated into the relational database.

{FIGURE PLACEHOLDER — Figure 3.7: A core entity-relationship diagram in crow's-foot notation. Central entities: COMPANIES (one-to-many into everything below); EMPLOYEES (company_id FK, department_id FK, location_id FK, self-referential manager_id FK); DEPARTMENTS (self-referential parent_department_id FK); LOCATIONS. Radiating from EMPLOYEES: ATTENDANCE_RECORDS; OVERTIME_REQUESTS; LEAVE_REQUESTS; LEAVE_BALANCES; TRANSITIONS (one-to-many TRANSITION_TASKS); EMPLOYEE_DOCUMENTS (holding r2_object_key); EMPLOYEE_ASSETS; LOANS (one-to-many LOAN_REPAYMENTS); PAYSLEIPS; ASSESSMENTS; PEER_REVIEWS; GOALS; BENEFIT_ENROLLMENTS. From COMPANIES: COMPANY_SETTINGS; PAYROLL_SETTINGS; TAX_BRACKETS; SALARY_COMPONENTS; PAY_GRADES; PAYROLL_RUNS (one-to-many PAYSLEIPS, one-to-many COMPLIANCE_TASKS); JOB_REQUISITIONS (one-to-many CANDIDATES, which is one-to-many INTERVIEWS, OFFERS and CANDIDATE_TIMELINE_EVENTS); REVIEW_CYCLES (one-to-many CYCLE_STAGES); ROLES; AUDIT_LOG; AI_QUERY_LOGS. Mark the company_id foreign key on every entity that carries one. Caption goes BELOW the figure.}

Figure 3.7: Core entity relationship diagram

Three design notes carry forward into Section 3.6. The self-referential manager reference on the employee record is what allows manager-scoped views and approval routing to be resolved without a separate reporting-structure table. The self-referential parent reference on departments supports arbitrarily nested organisational hierarchy. And the payroll run to payslip to compliance task chain is what makes a remittance obligation traceable back to the specific run that created it.

3.5.4 Request Pipeline Design

Section 3.2 presented the layered architecture; this section specifies the fixed sequence every request passes through before reaching module logic, since this is where the tenant and access layer of the conceptual framework is actually enforced.

The pipeline runs in five ordered stages. **Authentication** verifies the presented token's signature and, on success, places the employee identifier, company identifier, and role onto the request context. **Tenant resolution** establishes the company scope that every subsequent database operation will be constrained to. **Role gating** rejects the request if the authenticated role is not on the route's permitted list. **Permission narrowing** applies the custom permission matrix where the user has been assigned a custom role, and passes through unchanged where they have not. **Module dispatch** hands the request to its controller.

A rejection at any of the first four stages short-circuits the pipeline before module code executes. This is what makes NFR5's cross-tenant guarantee enforceable in one place rather than separately in every module, and it is the reason no module handler contains its own tenant or role check.

One qualification is necessary for accuracy. Manager-scoped filtering, restricting a manager to their own direct reports, is not enforced by this pipeline. It is applied inside the relevant service methods, which compare each row's manager reference against the caller's own identifier. The pipeline establishes *which company and which role*; the service establishes *which subset within that role*. This split is deliberate, because the direct-report relationship is data rather than identity and cannot be evaluated without querying, but it does mean the guarantee is enforced consistently by convention across services rather than structurally in one location.

{FIGURE PLACEHOLDER — Figure 3.8: A horizontal flowchart of the request pipeline. Boxes left to right: "Incoming request" → "Authenticate (verify token signature)" → "Resolve company" → "Check role against route allow-list" → "Apply custom permission narrowing" → "Dispatch to module controller" → branching to "D1" and "R2". Each of the four middle stages should have a downward arrow to a shared rejection box labelled "403 / 401 — request terminated before module logic". Caption goes BELOW the figure.}

Figure 3.8: Request processing pipeline within the Worker

3.6 Database Design

3.6.1 Multi-tenancy and tenant isolation strategy

Section 2.6 established three positions on the isolation spectrum: the silo model of one database per tenant, the bridge model of one schema per tenant within a shared database, and the pool model of one shared schema with a tenant discriminator column on every tenant-owned row. Hattab and Belalem (2023) characterise the choice as a trade-off between resource sharing and isolation strength, and their analysis makes clear that the trade-off is structural rather than incidental, so the position taken must be stated and defended.

This system adopts the pool model. All companies share one D1 database and one schema, and every tenant-owned row carries a company reference. Four reasons drive the choice, each tied to a requirement in Section 3.4.

Operational simplicity at the target tenant scale (NFR16, NFR17). A schema change is applied once, as a single versioned migration against a single database. Under a silo model the same change must be applied reliably to every tenant database, and a partial failure leaves the estate in mixed states. For a platform serving small and medium-sized organisations, where the operational capacity available is precisely what Destriani et al. (2024) identify as scarce, a migration path that cannot partially fail is a substantive advantage rather than a convenience.

Onboarding without provisioning (NFR16). Registering a new company inserts rows. It does not create, configure, and migrate a new database, so a tenant can be onboarded without a deployment and without any provisioning step that could fail midway.

Cost proportional to data rather than to tenant count. Each tenant in a silo model carries a fixed schema overhead regardless of how few employees it has. For a platform whose expected tenants are small, that overhead is charged repeatedly against tenants who use very little of it.

Cross-tenant reporting remains possible. Platform-level operational queries, spanning tenants, are a single query rather than a fan-out across databases.

The trade-off this accepts is stated plainly. Isolation is logical rather than physical. A query that omits the company filter would return rows belonging to other companies, so the guarantee in NFR5 rests on correct query construction rather than on physical separation. This is the pool model's known weakness, and it is the reason NFR6 exists as a separate requirement: the mitigation is to make the tenant filter structural rather than discretionary, by enforcing it in a shared data-access wrapper that injects the company constraint into every query, rather than requiring each service author to remember it. The project's engineering conventions accordingly treat an unscoped query as a security defect rather than a logic error. Section 4.2 reports how this wrapper is implemented, and Section 4.3 reports the cross-tenant access tests written specifically to verify it.

Table 3.2: Tenant isolation models evaluated for this system

Criterion	Silo (database per tenant)	Bridge (schema per tenant)	Pool (shared schema) — adopted
Isolation strength	Physical	Logical, schema-bounded	Logical, query-bounded
Tenant onboarding	Provision and migrate a database	Create and migrate a schema	Insert rows
Migration risk	Partial failure across estate	Partial failure across schemas	Single atomic migration
Fixed cost per tenant	High	Moderate	Negligible

Cross-tenant reporting	Fan-out databases	across	Fan-out schemas	across	Single query
Principal risk	Operational drift between tenants	Schema divergence	An unscoped query leaks data		

3.6.2 Schema conventions

Three conventions apply across the schema. Every tenant-owned table carries a company reference and creation and update timestamps. Primary keys are opaque identifiers rather than sequential integers, so that a record identifier appearing in a URL discloses nothing about record volume and cannot be enumerated. A small number of relationships, notably the employee-to-manager and department-to-manager references, are held as plain references rather than enforced foreign key constraints, to permit the ordering flexibility required when an organisational structure is being built up or restructured; this is a documented convention rather than an omission.

3.6.3 Employees table

The employee record is the entity every other module references. Its principal columns are set out below.

Table 3.3: Principal columns of the employees table

Column	Type	Notes
id	TEXT	Primary key, opaque identifier
company_id	TEXT	Foreign key to companies; the tenancy discriminator (FR6)
department_id	TEXT	Foreign key to departments
location_id	TEXT	Foreign key to locations

first_name, last_name, email	TEXT	Identity and login credential
password_hash, password_salt	TEXT	Salted hash only; never recoverable (NFR9)
is_password_changed	BOOLEAN	Drives the forced reset flow (FR3)
role	TEXT	One of six fixed roles; drives route gating (FR4)
custom_role_id	TEXT, nullable	Optional narrowing permission matrix (FR5)
manager_id	TEXT, nullable	Self-referential; resolves reporting lines and approval routing (FR9)
hire_date	DATE	Drives payroll proration (FR34)
annual_salary	DECIMAL	Basis for gross pay computation
status	TEXT	onboarding / active / on leave / separated (FR15)
bank_account, pension_id, pfa_name, tin, nin	TEXT, nullable	Statutory and disbursement identifiers; absence drives the pre-run exception list (FR42)

The self-referential manager reference is the single most structurally significant column outside the tenancy key. It resolves manager-scoped leave and attendance views, routes approval tasks, scopes team goal assignment, and determines which self-assessments a given manager may rate, all without a separate hierarchy table.

3.6.4 Attendance and overtime tables

Attendance stores one row per clock-in event rather than one row per day, so that a correction or a second session remains auditable rather than overwriting the original record (FR16).

Table 3.4: Principal columns of the attendance_records table

Column	Type	Notes
id	TEXT	Primary key
company_id, employee_id	TEXT	Tenancy and ownership
work_date	DATE	The day the session belongs to
clock_in, clock_out	TIMESTAMP	Clock_out nullable while a session is open
latitude_in, longitude_in	REAL, nullable	Geolocation at clock-in (FR17)
latitude_out, longitude_out	REAL, nullable	Geolocation at clock-out
location_label	TEXT, nullable	Free-text location description
work_hours, overtime_hours	DECIMAL	Computed per record (FR18)
status	TEXT	Classification against configured hours and grace period (FR19)

Overtime requests are held separately rather than as a flag on the attendance record, because an overtime claim has its own lifecycle: it is submitted with a justification, reviewed

by a manager, and approved or rejected independently of whether the underlying attendance was recorded (FR20).

3.6.5 Leave management tables

Leave splits across two tables. Requests hold the application and its outcome; balances hold the entitlement.

Table 3.5: Principal columns of the leave_requests and leave_balances tables

Table	Column	Type	Notes
leave_requests	id	TEXT	Primary key
	company_id, employee_id	TEXT	Tenancy and ownership
	leave_type	TEXT	Matches a balance row's type
	start_date, end_date	DATE	Requested range
	days	DECIMAL	Computed from the range
	reason	TEXT	Employee justification
	attachment_key	TEXT, nullable	R2 object key for supporting documents
	status	TEXT	pending / approved / rejected (FR22)

	manager_comment	TEXT, nullable	Recorded with the decision
leave_balances	employee_id, leave_type	TEXT	Composite key
	company_id	TEXT	Tenancy
	total_entitlement	DECIMAL	Allocation for the period (FR21)

The design decision worth stating here is what the balance table does *not* hold. It stores entitlement only. Days taken are computed at query time by summing approved requests, rather than maintained as a separately mutated running total (FR23). The alternative, decrementing a stored balance on each approval, introduces a class of defect this design eliminates entirely: if an approval is later reversed, a request is amended, or a write partially fails, a stored balance and the underlying request history can silently disagree, and there is no way to tell which is correct. Computing on read costs a small amount of query work in exchange for making that divergence structurally impossible.

3.6.6 Payroll tables

Payroll is the most structurally complex domain in the schema, spanning configuration, computation, and compliance.

Table 3.6: Payroll domain tables and their roles

Table	Role
payroll_settings	Per-company pay cycle, cut-off, payment day, and statutory rates (FR32)
tax_brackets	Per-company progressive PAYE bands, editable (FR39)
salary_components	Earnings and deductions catalogue, fixed or percentage-based (FR33, FR35)

pay_grades	Salary band definitions
loans	Active employee loans with instalment and outstanding balance (FR40)
loan_repayments	One row per repayment applied, generated when a run is marked paid
payroll_runs	Run header with period, status, and approval trail (FR41)
payslips	One row per employee per run, holding the resolved computation
compliance_tasks	Remittance obligations generated with statutory due dates (FR41)

The payslip design carries one decision that matters for auditability. Each deduction category is stored as its own column rather than aggregated into a single total, and every figure is resolved and persisted at computation time rather than recalculated when the payslip is read. Section 2.8 established why: a deduction that cannot be decomposed cannot be defended when an employee or a revenue authority disputes it, and a figure recalculated on read would silently change if a rate were subsequently edited, making a historical payslip unreproducible. Persisting the resolved values fixes each payslip as a record of what was actually computed under the rates in force at the time.

Table 3.7: Principal columns of the payslips table

Column	Type	Notes
id	TEXT	Primary key
company_id, payroll_run_id	employee_id, TEXT	Tenancy, ownership, and run linkage

period	TEXT	Pay period identifier
gross_pay	DECIMAL	After proration (FR34)
basic_salary, allowances	DECIMAL	Component split (FR35)
paye	DECIMAL	Computed per FR39
pension_employee, pension_employer	DECIMAL	Employee figure deducted; employer figure reported only (FR36)
nhf	DECIMAL	Deducted from net (FR37)
nsitf, itf	DECIMAL	Employer-borne; reported, never deducted (FR38)
loan_deduction	DECIMAL	Per FR40
other_deductions	DECIMAL	Manual entries
net_pay	DECIMAL	Final payable figure
benefits_snapshot	TEXT (JSON)	Allowance and pension figures as at computation time

3.6.7 Statutory computation design

The payroll engine's computation sequence is fixed and is documented here because it constitutes the core algorithmic contribution of the system.

Gross pay is derived from annual salary divided across the periods in the year and prorated where the employee's hire date falls within the period. Gross is split into basic salary and

allowances per the component catalogue. Pension is computed from the pensionable base, with the employee portion deducted and the employer portion computed and reported for remittance but never subtracted from the employee. The housing fund deduction is computed from basic salary. The two employer levies are computed from gross and tracked for compliance reporting without affecting net pay.

Income tax is computed on an annualised basis rather than monthly, because the Consolidated Relief Allowance and the progressive bands are defined annually and applying them to a monthly figure would produce a materially different and incorrect result. The annual relief is computed, subtracted from annual gross along with the annual pension contribution, and the resulting taxable income is run through the progressive bands, consuming each band's width in order so that bands can be re-ranged in configuration without breaking the computation. The annual tax figure is then divided across the periods in the year.

Loan deduction is applied at the lesser of the scheduled instalment or the outstanding balance, so that a nearly-repaid loan does not over-deduct. Net pay is gross less income tax, employee pension, housing fund, loan deduction, and any manual deductions.

{FIGURE PLACEHOLDER — Figure 3.9: A vertical flowchart of the gross-to-net payroll computation. Steps top to bottom: "Annual salary ÷ periods" → decision diamond "Hire date within period?" → yes branch "Prorate by days worked" → "Gross pay" → "Split into basic and allowances" → parallel branches for (a) "Employee pension — deducted", (b) "Employer pension — reported only", (c) "Housing fund from basic — deducted", (d) "Employer levies from gross — reported only" → "Annualise gross" → "Subtract Consolidated Relief Allowance and annual pension" → "Apply progressive bands in order" → "Divide annual tax across periods" → "Apply loan deduction: lesser of instalment or balance" → "Net pay". Shade the "reported only" branches differently from the "deducted" branches to make the employee-versus-employer attribution visually explicit. Caption goes BELOW the figure.}

Figure 3.9: Gross-to-net payroll computation sequence

Table 3.8: Statutory deduction treatment and remittance

Scheme	Computation base	Borne by	Effect on net pay	Remittance obligation generated
Pay-As-You-Earn income tax	Annualised gross less relief and pension	Employee	Deducted	Yes, with statutory due date
Pension — employee portion	Pensionable base	Employee	Deducted	Yes, with statutory due date

Pension employer portion	—	Pensionable base	Employee	None	Yes, with statutory due date
National Fund	Housing	Basic salary	Employee	Deducted	Yes, with statutory due date
Nigeria Insurance Fund	Social Trust	Gross pay	Employee	None	Yes, with statutory due date
Industrial Fund	Training	Gross pay	Employee	None	Yes, with statutory due date

The distinction in the third column is the one most frequently mishandled in manual payroll and is enforced structurally here: the employer-borne schemes are computed and stored, so they appear in remittance reporting, but they are excluded from the net pay subtraction by construction rather than by the operator remembering to exclude them.

3.7 System Interface Design

The interface translates the role model directly into what each user sees. Rather than presenting one interface with controls hidden or disabled according to role, each role is served a distinct set of views scoped to what NFR18 requires. Hiding a control is treated as a usability measure only; it is never the mechanism by which access is restricted, since every route independently re-validates authorisation server-side (NFR11).

HR Administrator and Super Administrator. A company-wide dashboard surfacing headcount, pending leave and overtime approvals, open requisitions, active transitions, and payroll run status, with drill-through into any module's configuration and any individual employee record. Super Administrators additionally reach the control centre.

Payroll Officer. A payroll-focused dashboard showing the current run's state, the pre-run exception list, and the compliance calendar of upcoming remittance deadlines, with access to run preview, submission, and reporting but not to unrelated HR modules.

Recruiter. A pipeline view organised by requisition and stage, with candidate detail, interview scheduling, scorecard entry, and offer management, plus funnel and time-to-hire analytics.

Manager. A dashboard scoped to direct reports: pending leave and overtime requests, team goals, self-assessments awaiting review, and peer-review nominations awaiting approval.

Employee. A self-service view limited to the user's own record: clock in and out, leave balance and application, payslips, assigned assets, benefit enrolment, assigned courses and surveys, self-assessment submission, and support tickets.

Across all roles the interface follows one pattern: a persistent navigation frame scoped to the modules the role can reach, a main panel showing either a list view or a single-record detail view, and modal or slide-over panels for actions not warranting a full page transition. Server state is managed through a query-caching layer that handles background refetching and cache invalidation on mutation, so that an approval performed in one view is reflected in dependent views without a manual refresh.

Navigation is handled predominantly through internal view state rather than URL routing, with one deliberate exception: the public careers page is reachable by a real URL, because an external applicant must be able to receive and open a link to a specific open role (FR31).

{FIGURE PLACEHOLDER — Figure 3.10: A set of three annotated interface wireframes side by side, showing the same navigation frame resolving differently by role. Left: HR Administrator dashboard, with a full module list in the sidebar and company-wide KPI cards. Centre: Manager dashboard, with a reduced sidebar and a pending-approvals list scoped to direct reports. Right: Employee self-service view, with a minimal sidebar and cards for clock status, leave balance, and latest payslip. Annotate each with a callout stating that the sidebar difference is a usability measure and that authorisation is separately enforced server-side. Caption goes BELOW the figure.}

Figure 3.10: Role-scoped interface views

3.8 Security Design

Security in this system is enforced at four points already established: the request pipeline (Section 3.5.4), the tenant scoping applied at the data-access layer (Section 3.6.1), the audit mechanism, and the tool boundary around the assistant. This section consolidates those into a single security design and states plainly what is, and is not, in scope.

3.8.1 Authentication

Authentication is password-based, with credentials stored only as salted hashes (NFR9). Successful authentication issues a signed token carrying the employee identifier, company identifier, and role. The token is verified at the first stage of the request pipeline before any tenant or role resolution occurs. Because the token is signed rather than merely encoded, its claims cannot be modified by the client without invalidating the signature, which is what allows the company claim to be trusted as the basis for tenant scoping (Dalimunthe et al., 2023).

3.8.2 Authorisation

Authorisation operates in two composable layers, as specified in FR4 and FR5. The coarse layer gates each route against an allow-list of fixed roles. The fine layer applies a custom permission matrix where one has been assigned to the user, and is strictly narrowing: it can

remove access that the fixed-role gate has granted but can never grant access beyond what that gate permits. A user with no custom role assigned passes through the second layer unchanged.

This ordering is deliberate. Because narrowing can only subtract, a misconfigured custom role can lock a user out of something they should reach, which is visible and recoverable, but cannot grant a user access to something their fixed role never permitted, which would be neither.

The frontend hides controls a role cannot use, but every route re-validates the role and company independently, so a manipulated client request that skips a hidden control is rejected at the pipeline rather than reaching module logic (NFR11).

3.8.3 Tenant isolation

As set out in Section 3.6.1, tenant isolation under the pool model is logical and therefore depends on every query carrying the company constraint. The design response is to make that constraint structural: the company filter is injected by a shared data-access wrapper rather than written by each service author, so that omitting it requires actively bypassing the wrapper rather than merely forgetting a clause. Cross-tenant access attempts are written as explicit test cases rather than assumed, and are reported in Section 4.3.

3.8.4 Data in transit and at rest

All communication between the client, the Worker, and the platform data services is encrypted (NFR8). The Worker reaches D1, R2, and the inference service through platform bindings rather than a public network path. Files in object storage are not exposed through public or long-lived URLs; retrieval is mediated by a route that re-validates the requester's role and company on every access.

3.8.5 Audit design

The audit mechanism records the actor, action, module, severity, descriptive detail, and originating IP address for sensitive administrative operations, and is invoked from the settings, company profile, department, location, role, holiday, email template, integration, workflow, and data export paths (FR60). The log is queryable and exportable, and per-employee audit history is additionally surfaced on the employee record.

The immutability requirement in NFR12 is enforced structurally rather than by convention: the audit table is written through an insert-only path, and no route in the system exposes an update or delete operation against it. An audit trail therefore cannot be edited through the application even by an account holding the highest privilege level.

3.8.6 AI assistant security boundary

The assistant module raises a security problem that the rest of the system does not, and Section 2.9 established why: a language model does not inherit the application's authorisation logic, and the leading catalogued risks for such systems, prompt injection and

excessive agency, concern precisely the boundary between the model and the data it can reach (OWASP, 2025).

The design response has five components, each corresponding to a functional requirement.

The model is given tools, not data. It has no database access. It can only invoke a fixed set of named, read-only functions (FR62). There is no tool capable of creating, modifying, or deleting a record, so a successful injection cannot produce a write.

Authorisation is re-derived inside each tool, not inferred from the model's arguments (FR63). If the model requests an employee record, the tool implementation checks the caller's role and, where the tool is manager-scoped, cross-references the caller's own direct-report set before answering. A request for out-of-scope data returns a structured refusal rather than the data, and the system instruction directs the model to relay that refusal rather than attempt an alternative route to the same information. This is the central design principle: the model is treated as an untrusted natural-language front end over a pre-authorised capability set, never as an authority on what it may see.

Aggregation replaces detail where detail is not required. The payroll tool returns aggregate totals rather than individual salaries, so that a question the model can legitimately answer does not require exposing figures the caller has no need to see.

The agent loop is bounded (FR64). After a fixed number of tool-calling iterations the loop terminates with a request to narrow the question, which bounds both cost and the scope of any single interaction.

Every query is logged (FR66), recording the company, employee, role, question, and tools invoked, which provides both an operational usage record and evidence of correct scoping behaviour.

{FIGURE PLACEHOLDER — Figure 3.11: A sequence diagram of an assistant request. Participants left to right: Employee, Worker (assistant route), Inference service, Tool implementation, D1. Sequence: (1) Employee sends question with token; (2) Worker verifies token and extracts identity, company, role; (3) Worker sends prompt plus tool definitions to inference service; (4) Inference service returns a tool call with arguments; (5) Worker invokes the tool implementation; (6) Tool implementation re-derives authorisation from the verified session, NOT from the model's arguments — annotate this step prominently; (7) branch: if authorised, query D1 and return data; if not, return a structured refusal object; (8) Worker returns tool result to inference service; (9) loop back to step 4 up to the iteration bound; (10) Inference service returns final answer; (11) Worker writes the query log; (12) Worker returns answer to Employee. Caption goes BELOW the figure.}

Figure 3.11: AI assistant request sequence with authorisation re-derivation

Table 3.9: Assistant tool set and authorisation rules

Tool	Returns	Authorisation rule applied within the tool
------	---------	--

Get employee	A single record	employee	Self, or a direct report, or an administrative role
Search employees	Matching summaries	employee	Company-scoped; administrative roles only for full directory
Get leave balance	Entitlement taken	and days	Self, or a direct report, or an administrative role
Get pending leave requests	Outstanding requests		Manager's own direct reports, or company-wide for administrative roles
Get headcount	Aggregate count	employee	Company-scoped
Get payroll summary	Aggregate totals only	payroll	Administrative and payroll roles; never individual salary figures
Get open requisitions	Open roles		Company-scoped
Get compliance tasks due	Upcoming obligations	remittance	Administrative and payroll roles

3.8.7 Documented deviation: header-based tenant fallback

One deviation from the intended authentication model exists in the current implementation and is documented here rather than omitted, because an accurate account of a system's security posture is more useful than a flattering one.

The authentication middleware accepts, on most administrative routes, a company identifier supplied directly in a request header as a substitute for a verified token. Where no token is present, this header establishes the company scope for the request. The mechanism originates from an earlier development phase that preceded the completion of token-based authentication, when the frontend operated against seeded data and required a tenant context without an authentication flow to supply one.

Its security implication is stated plainly. A request presenting only this header is scoped to a company but carries no verified identity and no role claim. The role-gating stage of the pipeline therefore has nothing authoritative to evaluate, which means the fine-grained authorisation guarantees described in Section 3.8.2 do not hold for such a request on the routes where the fallback is accepted. It is a genuine weakening of the access-control model, not a theoretical one.

Three qualifications apply. First, the fallback is explicitly rejected at the assistant endpoint (FR65), because the tool-scoping logic described in Section 3.8.6 keys off a role claim that a header-only request does not carry; a request without a verified token is refused there rather than served with reduced scoping. Second, tenant isolation itself is not bypassed by the fallback: a request is still constrained to one company, so the mechanism does not permit cross-tenant retrieval and NFR5 is not defeated. Third, the production client authenticates by token on every request, so the fallback is a latent capability of the server rather than a path in active use.

It nonetheless remains a defect. It is recorded among the system's limitations in Chapter Five, and its removal is the first of the recommendations offered there. A related hardening item is recorded alongside it: the token signing secret falls back to a fixed development value when the environment variable is unset, which must be explicitly configured in any production deployment.

3.8.8 Scope boundaries

Two boundaries stated in Section 1.4 are restated here because they bear directly on how this chapter's claims should be read.

Alignment, not certification. The controls described above, least-privilege access, immutable audit logging, encrypted transport, and scoped file retrieval, are designed with recognised information-security control principles in mind. This project does not pursue or claim formal certification against any standard, which would require an external audit process outside an academic project's scope. Chapter Four accordingly describes these controls as functioning as designed, not as independently certified.

No third-party security assessment. The controls are validated through the project's own functional and unit testing, including the cross-tenant access tests noted in Section 3.8.3. No penetration test or external security audit has been conducted, so Chapter Four's security-relevant testing should be read as verifying that the described controls behave as specified, not as a comprehensive assessment of the system's resistance to attack.

CHAPTER FOUR: SYSTEM IMPLEMENTATION AND DISCUSSION

4.1 Introduction

This chapter reports how the system designed in Chapter Three was implemented, tested, and evaluated. Section 4.2 describes the development environment, toolchain, and operating requirements. Section 4.3 works through the implementation of each functional module, covering the backend routes, the data model as realised in code, and the frontend views that consume them. Section 4.4 reports the testing strategy and its results. Section 4.5 discusses those results against the objectives set out in Section 1.3, presenting the response-time evaluation and the cost-model analysis that together discharge Objective 8, and reporting the platform constraints the evaluation surfaced.

The account given here describes the system as built. Where the implementation departs from the design in Chapter Three, or where a platform constraint limits what the design can achieve, the divergence is reported rather than omitted.

4.2 System Development Environment

4.2.1 Backend runtime and framework

The backend runs on Cloudflare Workers and is built with Hono, a lightweight web framework designed for edge runtimes. Development and testing were carried out locally using Wrangler, the platform's command-line tool, which runs the same underlying runtime locally through a local emulation layer rather than a separate development server. This matters for fidelity: the local environment executes the same isolate-based runtime as production, so behaviour that depends on runtime characteristics does not diverge between the two. Local iteration used the Wrangler development server, and publication to the production edge network used the Wrangler deploy command.

Hono was selected over heavier alternatives because Workers imposes a bundle-size constraint that conventional Node.js frameworks do not comfortably meet, and because its middleware model maps directly onto the request pipeline specified in Section 3.5.4: authentication, tenant resolution, role gating, and permission narrowing compose as ordered middleware rather than as checks scattered through handlers.

4.2.2 Database and object-relational mapping

The database is Cloudflare D1, accessed through Drizzle ORM's D1 driver. The schema is defined declaratively in TypeScript as typed table definitions rather than as hand-written SQL, distributed across model files by functional domain and aggregated into a single schema barrel.

This arrangement delivers a property worth stating explicitly, because it is a genuine implementation benefit rather than a generic tooling note: because the table definitions are TypeScript types as well as schema declarations, every query built against them is type-checked at compile time. A column renamed in the schema produces a compile error at each query that references it, rather than a runtime failure discovered in production. For a schema of this size, this is the difference between a schema change being a mechanical refactor and being a source of latent defects.

Migrations were generated from schema changes using Drizzle Kit, applied to a local D1 instance during development and to the production database once verified. Migrations are sequential, versioned, and never hand-edited, satisfying NFR17. Forty-six migrations were produced over the development period, numbered from the initial schema through to the final feature additions.

4.2.3 Object storage and inference

Object storage uses Cloudflare R2, bound to the Worker through the same configuration file as the database and accessed through the platform's native binding API rather than through the ORM, since R2 is object storage rather than a relational store and Drizzle provides no driver for it.

The assistant module uses Cloudflare Workers AI for hosted inference, invoked from the same Worker as the rest of the API and therefore passing through the same authentication and tenant-resolution pipeline before any inference call is made.

4.2.4 Frontend

The frontend is a React single-page application built with Vite and written in TypeScript, deployed to Cloudflare Pages independently of the backend, with no shared deployment pipeline between the two. It communicates with the Worker API exclusively over HTTPS as a conventional REST client.

Server state is managed through TanStack Query rather than through component-local state or a global store. This choice does more work than it appears to: it provides caching, background refetching, and automatic cache invalidation on mutation, which means an approval performed in one view invalidates and refreshes the dependent views without explicit coordination between components. Charting uses Recharts, iconography uses Lucide, and transitions use Framer Motion.

The application uses internal view state for navigation rather than URL routing, with the single deliberate exception of the public careers page, which is reachable by a real URL because an external applicant must be able to open a link to a specific role.

4.2.5 Version control, secrets, and deployment

Source code is held in a Git repository hosted on GitHub, developed on a single main branch across eighty-seven commits at the time of writing.

Platform secrets, including the token signing key and service credentials, are managed through the Wrangler secret command, which stores values on the platform rather than in the repository. Local development secrets are held in a local variables file excluded from version control. If the signing secret is absent, the server refuses to issue or verify tokens rather than falling back to a default value (Section 3.8.7), so a deployment cannot run with a predictable key by omission.

Deployment is manual. The backend publishes through the Wrangler deploy command and the frontend through a build step followed by a Pages deployment command. No continuous integration pipeline is configured, so there is no automated build or test gate ahead of a release. This is recorded as a limitation in Section 1.4 and revisited in Chapter Five.

4.2.6 Hardware and software operating requirements

Development environment. A development machine requires only Node.js at version 16.17.0 or later, a Git client, and a Cloudflare account with Workers, D1, R2, and Workers AI enabled. No local database server, container runtime, or web server is required, since the platform's local emulation supplies all of them.

Production environment. There is no provisioned hardware requirement. No servers are sized, purchased, or configured; no operating system is installed or patched; no database instance is provisioned. The production requirement is an active Cloudflare account with the relevant services enabled. This is not merely a convenience of the write-up: the absence of a production hardware specification is the direct, observable consequence of the architecture argued for in Chapters One and Two, and it is the clearest single demonstration of the operational-overhead claim made in Section 1.6.

Client environment. Any modern browser supporting current JavaScript standards. No client installation is required.

Table 4.1: Development toolchain and platform services

Layer	Technology	Role
Frontend framework	React	Component-based user interface
Frontend build	Vite	Development server and production bundler
Server-state management	TanStack Query	Caching, refetching, cache invalidation

Language	TypeScript	Static typing across frontend and backend
Backend runtime	Cloudflare Workers	Serverless edge compute
Backend framework	Hono	Routing and middleware composition
Object-relational mapper	Drizzle ORM	Typed schema definition and query building
Migrations	Drizzle Kit	Schema-diff migration generation
Database	Cloudflare D1	Serverless SQLite
Object storage	Cloudflare R2	Document and file storage
Inference	Cloudflare Workers AI	Hosted language-model inference
Static hosting	Cloudflare Pages	Frontend asset delivery
Local emulation	Wrangler	Local runtime, database, and deployment
Testing	Vitest	Unit and integration testing
Version control	Git and GitHub	Source management

4.3 Implementation of the HRMS

The implementation is organised by module, mirroring the design in Chapter Three. Each subsection covers the backend routes and services, the schema as realised, and the frontend views that consume them.

The API is organised under six top-level route groups mounted on the application: authentication, public, employee, administrative, assistant, and support. The administrative group is itself composed of sub-routers for payroll, leave, requisitions, applicant tracking, attendance, and benefits, producing a predictable resource-oriented URL scheme. Across all groups the implementation exposes close to three hundred endpoints.

4.3.1 Tenancy, authentication, and access control

This module implements FR1 to FR6 and sits beneath every other module as the tenant and access layer described in Section 3.3.

Tenant provisioning (FR1). A new company is registered through a route that inserts the company record and its first administrator account. Consistent with the pool isolation model adopted in Section 3.6.1, this is an insert operation. No database is created, no schema is migrated, and no deployment occurs, which is what satisfies NFR16's requirement that a tenant be onboardable without a code change. The company's payroll settings, salary component catalogue, statutory rates, and progressive tax bands are not written at registration. Each is created from defaults defined in the payroll service the first time it is read, so registration remains two inserts and a tenant that never runs payroll stores no payroll configuration.

Authentication (FR2, FR3). Login accepts an email address and password, retrieves the stored salt and hash, recomputes the hash, and compares. On success it issues an HS256-signed token carrying the employee identifier, company identifier, and role. The token is held client-side and presented as a bearer credential on subsequent requests. The temporary password reset flow sets a flag on the employee record that forces a password change at next login before any other view is reachable.

Tenant scoping (FR6, NFR6). This is the implementation detail that carries the most security weight in the system, because the pool model makes correct scoping the sole barrier to cross-tenant disclosure. Rather than requiring each service method to remember a company filter, the implementation routes queries through a shared data-access wrapper that injects the company constraint from the request context into every query it builds. Omitting the constraint therefore requires actively bypassing the wrapper rather than merely forgetting a clause. The project's engineering conventions treat an unscoped query as a security defect rather than a logic error, and Section 4.4.3 reports the tests written specifically to verify the guarantee holds.

Role gating and permission narrowing (FR4, FR5). Route-level guards accept an allow-list of permitted roles, and a shared authorisation middleware performs a constant-time lookup against the role extracted from the verified token before the handler executes. Where a user has been assigned a custom role, a second middleware applies that role's permission matrix over the defined modules and actions. The second layer is strictly narrowing, as specified in Section 3.8.2; a user with no custom role assigned passes through unchanged.

Organisational hierarchy (FR8). Departments carry a self-referential parent reference supporting arbitrarily nested structures. The frontend renders the hierarchy as an expandable tree.

Figure 4.1 shows the two entry points this module governs: the credential prompt and the forced reset that an administrator-created account encounters before any other view becomes reachable.

{FIGURE PLACEHOLDER — Figure 4.1: Two screenshots side by side. Left: the login screen showing the email and password fields. Right: the forced password-reset screen presented at first login to a user whose password has not yet been changed, with the main navigation visibly unavailable behind it. Caption goes BELOW the figure.}

Figure 4.1: Login and forced password change on first sign-in

4.3.2 Workforce and organisational management

This module implements FR7 to FR11 and provides the employee directory and the consolidated employee profile.

The routes support full lifecycle operations on employee records: creation with employment details and statutory identifiers, retrieval with search by name, update, and status change. Related records are managed alongside: emergency contacts, the per-employee document store, and department and location assignment. A temporary password reset route is exposed to administrators. The directory endpoint returns a company's full employee list in a single response rather than paginating on the server. At the headcounts evaluated in Section 4.5.1 this is inexpensive, but response size grows linearly with headcount, and server-side pagination is noted in Chapter Five as a refinement for larger tenants.

The employee profile view assembles data from several modules into one screen, presenting employment details, assigned assets, leave balance, recent attendance, current goals, and the audit history for that employee's record. This is the one place in the frontend where a view deliberately spans module boundaries, and it is implemented as parallel queries against each module's endpoint rather than as a single aggregating endpoint, so that no module's failure prevents the rest of the profile from rendering.

Organisational structure, covering departments and locations and their membership, is managed through a dedicated service.

Figure 4.2 shows the directory and the consolidated profile view described above, including the audit history that surfaces changes made to an individual record.

{FIGURE PLACEHOLDER — Figure 4.2: Two screenshots. Left: the employee directory with the name search control visible and several records listed. Right: a single employee profile assembling employment details, assigned assets, leave balance, recent attendance, and the audit history tab for that record. Caption goes BELOW the figure.}

Figure 4.2: Employee directory and consolidated employee profile

4.3.3 Onboarding and offboarding transitions

This module implements FR12 to FR15 through a generic transition model that represents either an onboarding or an offboarding journey for one employee.

A transition comprises a checklist of tasks, each categorised by the function responsible for it, and each carrying an assignee, a due date, and a completion state. The design decision that carries weight here, and which directly addresses the cross-functional workflow weakness identified in Section 2.3, is that a transition's overall stage is **derived** from the completion state of its tasks rather than being independently editable (FR13). It is therefore structurally impossible for a case to be recorded as complete while a required sign-off remains outstanding, which is exactly the failure mode that manual, email-tracked clearance processes exhibit.

Offboarding additionally captures the separation reason, the handover recipient, and whether an exit interview is scheduled.

A new employee created with onboarding status is routed at first login into a dedicated self-service onboarding flow rather than the standard portal (FR15), which presents the tasks assigned to them and blocks progression to the main application until the required items are complete.

The frontend renders the onboarding dashboard with conditional display of the current stage, and renders offboarding as a clearance board showing each responsible function's sign-off status, updating as each is submitted.

Figure 4.3 shows an offboarding case mid-clearance. Note that the stage indicator visibly lags the board while sign-offs remain outstanding, because stage is derived from task completion rather than being independently editable.

{FIGURE PLACEHOLDER — Figure 4.3: An offboarding clearance board showing each responsible function's sign-off status, with some functions signed off and at least one still outstanding, and the derived stage indicator visible so that it can be seen to lag the incomplete board. Caption goes BELOW the figure.}

Figure 4.3: Offboarding clearance board showing per-function sign-off

4.3.4 Attendance and overtime

This module implements FR16 to FR20.

Clock-in and clock-out are separate operations writing to an attendance record that permits multiple sessions within a single day rather than overwriting a prior entry (FR16), so that a correction or a second session remains auditable. Where the client supplies them, latitude and longitude are captured at both clock-in and clock-out alongside an optional free-text location label (FR17). Worked hours and overtime hours are computed per record at clock-out (FR18).

Classification against the company's configured working hours, grace period, and working-day calendar is applied at clock-in (FR19), with the configuration read from company settings rather than hard-coded, so that each tenant's policy applies independently as NFR7 requires.

Overtime requests are implemented as a separate resource with their own approval lifecycle (FR20): an employee submits hours worked outside a scheduled shift with a justification, and a manager approves or rejects. Manager visibility is filtered inside the service by comparing each record's manager reference against the caller's own identifier, consistent with the qualification stated in Section 3.5.4.

Figure 4.4 shows the employee-facing capture interface alongside the administrative register that classification feeds.

{FIGURE PLACEHOLDER — Figure 4.4: The attendance view immediately after a clock-in, showing the punctuality classification applied against the company's configured start time and grace period, together with the captured location where the browser granted permission. Caption goes BELOW the figure.}

Figure 4.4: Attendance clock-in with punctuality classification

4.3.5 Leave management

This module implements FR21 to FR24.

Leave balances hold entitlement per employee per leave type. Days taken are computed at query time by summing approved requests rather than maintained as a separately mutated running total (FR23). Section 3.6.5 set out why: computing on read makes divergence between a stored balance and the underlying request history structurally impossible, at the cost of a small amount of query work. In practice the query cost is negligible, because the summation is bounded by one employee's requests within one leave year.

Requests carry a date range, computed day count, reason, optional attachment reference, status, and the manager's comment recorded with the decision. Managers see only their direct reports' requests, filtered in the service layer; administrators see the company-wide queue through a dedicated approvals screen.

Figure 4.5 shows the employee application view and the manager queue it feeds, with balances computed from approved requests as described above.

{FIGURE PLACEHOLDER — Figure 4.5: Two screenshots. Left: the employee leave application form showing entitlement and days taken by leave type. Right: the manager approval queue scoped to direct reports, showing pending requests with approve and reject actions and the comment field recorded with the decision. Caption goes BELOW the figure.}

Figure 4.5: Leave balances and request approval

4.3.6 Recruitment and applicant tracking

This module implements FR25 to FR31 across two connected layers.

Requisitions. A job requisition carries an approval workflow moving through pending approval, open, and then one of on hold, filled, cancelled, or rejected. It records the requester, the reviewer, the budget range, and the business justification (FR26), together with optional public-facing copy and a flag controlling whether the role appears on the careers page.

Applicant tracking. A candidate progresses through applied, screening, interview, offer, and then hired or rejected. Interviews record per-interviewer scorecards covering assessment dimensions and a hire or no-hire recommendation, rather than a single shared verdict, which supports panel interviews where individual assessments should be recorded before being reconciled (FR28). Every stage transition, interview, and offer event is appended to a per-candidate timeline (FR30), producing a complete record of how a hiring decision was reached.

Public careers page. An unauthenticated route serves publicly flagged open requisitions for a given company and accepts applications, creating a candidate record tagged with its source (FR31). This is the only route in the application reachable by a real URL rather than internal navigation state.

One implementation decision is worth recording explicitly because it is a deliberate omission rather than an unbuilt feature. The module performs **no automated candidate ranking, résumé scoring, or algorithmic screening**. Section 2.9 set out the basis: Mori et al. (2025) document evidence that selection algorithms trained on unrepresentative data produce systematically unfair outcomes for candidates in protected subgroups, and that candidates prefer retaining human involvement in evaluation. The design position adopted is that the system should improve the structure, consistency, and auditability of a human hiring decision rather than substitute for it, and the scorecard-plus-timeline implementation is what that position looks like in code.

Figure 4.6 shows the pipeline and the per-candidate record that accumulates behind it. Figure 4.7 shows the single unauthenticated surface in the application.

{FIGURE PLACEHOLDER — Figure 4.6: Two screenshots. Left: the recruitment pipeline showing candidates grouped by stage from applied through hired. Right: a candidate detail view showing the timeline of stage transitions, interviews, and offer events, together with at least two completed interviewer scorecards so that per-interviewer assessment rather than a single shared verdict is visible. Caption goes BELOW the figure.}

Figure 4.6: Recruitment pipeline and interviewer scorecard

{FIGURE PLACEHOLDER — Figure 4.7: The public careers page as an external applicant sees it, showing publicly flagged open roles and the application form. Retain the browser address bar in this figure only, since this is the single route in the application reachable by a real URL and the address bar is the point. Caption goes BELOW the figure.}

Figure 4.7: Public careers page for an open requisition

4.3.7 Payroll and statutory compliance

This module implements FR32 to FR42 and is the most computationally significant part of the system. Every figure is derived from live employee, attendance, and loan data; nothing is re-keyed.

Per-employee computation. The engine computes a payslip per employee per period following the sequence specified in Section 3.6.7. Gross pay derives from annual salary divided across the periods in the year, prorated where the hire date falls within the period (FR34). Gross is split into basic salary and allowances per the configured component catalogue (FR35). Employee pension is computed from the pensionable base and deducted; the employer contribution is computed and reported for remittance but never subtracted from the employee (FR36). The housing fund deduction is computed from basic salary (FR37). The two employer levies are computed from gross and tracked for compliance reporting while being excluded from the net pay subtraction by construction (FR38).

Income tax is computed on an annualised basis (FR39). The Consolidated Relief Allowance is computed and subtracted from annual gross along with the annual pension contribution, and the resulting taxable income is run through the company's progressive bands. The bands are consumed in order, each absorbing its own width before the next applies, which is what allows an administrator to re-range the bands in configuration without breaking the computation. The annual figure is then divided across the periods in the year. Annualising rather than computing monthly is not a stylistic choice: the relief allowance and the bands are defined annually, and applying them to a monthly figure yields a materially different and incorrect result.

Loan deduction applies the lesser of the scheduled instalment or the outstanding balance, so a nearly-repaid loan does not over-deduct (FR40). Net pay is gross less income tax, employee pension, housing fund, loan deduction, and manual deductions.

Run lifecycle (FR41). A run moves through preview, submission, approval or rejection, and marking as paid. Preview recomputes live and persists nothing, so an operator can inspect the result before committing. Submission persists the run header and one payslip row per employee. Marking a run as paid triggers three effects alongside the status change: each employee's active loan balance is decremented and a repayment record created, closing the loan once fully repaid; statutory remittance obligations are generated as dated compliance tasks; and the export files are made available.

The generation of remittance obligations is the part of this module that responds most directly to the gap identified in Section 2.8. The literature reviewed there automates computation but leaves filing deadlines to manual tracking. Here, finalising a run produces the corresponding remittance tasks with their statutory due dates attached, converting a reminder process into a tracked artefact of the run that created it.

Exports. The module produces a bank disbursement file and scheme-specific remittance files shaped to resemble each agency's filing format. The system computes and exports; it does not integrate with a payment rail and does not move funds, as stated in Section 1.4.

Pre-run exceptions (FR42). Before submission, the payroll dashboard surfaces an exception list flagging employees missing bank details, salary configuration, or statutory identifiers, so that a run is not submitted with records that will fail at disbursement or remittance.

Figure 4.8 shows the preview stage of the run lifecycle, where figures can be inspected before anything is persisted. Figure 4.9 shows the two artefacts a finalised run produces.

{FIGURE PLACEHOLDER — Figure 4.8: A screenshot of the payroll run preview showing per-employee gross pay, each deduction category as a separate column, and net pay, with the pre-run exception list displayed above or alongside it flagging employees missing bank details, salary configuration, or statutory identifiers. Caption goes BELOW the figure.}

Figure 4.8: Payroll run preview with pre-run exceptions

{FIGURE PLACEHOLDER — Figure 4.9: Two screenshots. Left: a generated payslip showing the decomposed deduction categories, with employee-borne and employer-borne items distinguishable. Right: the compliance calendar listing the remittance tasks generated by the run, each with its statutory due date. Caption goes BELOW the figure.}

Figure 4.9: Generated payslip and statutory compliance calendar

4.3.8 Performance management

This module implements FR43 to FR48.

A company-wide review cycle comprises named stages, each with its own start and due window, so that a cycle's sequence is configurable rather than fixed. Within an active cycle, employees submit self-assessments covering achievements, challenges, skill ratings, and development goals, which their manager subsequently rates and comments on.

Peer reviews require nomination and manager approval before the nominated reviewer can submit, which prevents unsolicited peer assessment. Upward reviews, in which a report reviews their manager, skip the nomination gate, since requiring a manager to approve who may review them would defeat the purpose.

Goals are scoped to an individual, team, department, or the company, carry progress tracking, and support linkage to a parent goal so that individual objectives roll up to organisational ones. Peer recognition entries provide a lightweight acknowledgement mechanism separate from formal review. Training records track assigned, in-progress, and completed items.

Figure 4.10 shows an active review cycle with its named stages and their start and due windows.

{FIGURE PLACEHOLDER — Figure 4.10: The review cycle administration view showing the named stages with their start and due windows and completion progress across the company. Caption goes BELOW the figure.}

Figure 4.10: Performance review cycle configuration

4.3.9 Benefits, wellness, learning, assets, surveys, and support

These modules implement FR49 to FR55 and are grouped here because each follows the same structural pattern of an administrator-configured catalogue with employee-facing participation.

Benefits and wellness. Administrators define benefit plans with employer and employee cost split and coverage limits; employees enrol and register dependents. Wellness programmes run with per-participant progress tracking. Insurance and wellness reimbursements share one claims workflow, and an approved wellness claim posts against the employee's wellness budget.

Assets. The registry records category, serial number, condition, purchase value, and assignment status, with a company-wide administrative view and a personal view of an employee's own assigned items.

Learning. Administrators create courses referencing external content and assign them to employees, producing enrolment records tracked through assigned, in progress, and completed with a percentage progress value.

Surveys. Administrators author surveys targeted company-wide or by department, composed of ordered rating, free-text, and multiple-choice questions. Responses may be anonymous, implemented by leaving the employee reference null on the response record rather than by obscuring it after the fact, so that anonymity is a property of what was stored rather than of what is displayed.

Support. An internal helpdesk provides categorised, prioritised tickets with a threaded message log shared between the employee and the responding administrator.

4.3.10 Control centre and audit

This module implements FR56 to FR60.

The control centre covers company profile, security settings, operational calendar including working days and public holidays, custom role and permission management, API key issuance, transactional email template editing, the integration catalogue, an automation catalogue, and a full tenant data export producing a structured dump of every table belonging to the company.

Two entries in the integration catalogue require an honest account. Only the messaging integration performs a live handshake, in which an administrator supplies a webhook URL and can send a test message with delivery status recorded. Every other catalogue entry is a configuration toggle with no execution behind it, and the automation catalogue defines named step sequences without an execution engine. These are recorded as limitations in Section 1.4 rather than presented as working integrations.

Audit (FR60). The audit service is invoked from the settings, company profile, department, location, role, holiday, email template, integration, workflow, and data export paths, recording the actor, action, module, severity, descriptive detail, and originating IP address taken from

the platform's connecting-IP header. The log is searchable, filterable, and exportable, and per-employee audit history is surfaced on the employee profile.

The immutability requirement in NFR12 is enforced structurally: the audit table is written through an insert-only path, and no route exposes an update or delete operation against it. An audit entry therefore cannot be altered through the application by any account, regardless of privilege.

4.3.11 AI assistant

This module implements FR61 to FR66 and is the project's applied-AI contribution.

The assistant is built on Cloudflare Workers AI using a hosted instruction-tuned model with native function-calling support. Its architecture follows one governing principle established in Section 3.8.6: **the model is never trusted to enforce access control, and every tool re-derives its own authorisation from the caller's verified identity.**

Tool set (FR62). Eight read-only tools are exposed, covering employee retrieval, employee search, leave balance, pending leave requests, headcount, payroll summary, open requisitions, and compliance tasks due. There is no tool capable of creating, modifying, or deleting a record, so a successful prompt injection cannot produce a write. The payroll tool returns aggregate totals only and never individual salary figures, so a question the model can legitimately answer does not require exposing data the caller has no need to see.

Authorisation re-derivation (FR63). Each tool implementation independently checks the caller's role and, for manager-scoped tools, cross-references the caller's direct-report set before answering, regardless of what the model supplied as arguments. A tool that would expose out-of-scope data returns a structured error object rather than the data, and the system instruction directs the model to relay that refusal plainly rather than attempt an alternative route to the same information. This is the concrete implementation of the mitigation Section 2.9 identified as architectural rather than prompt-based: because the check happens inside the tool rather than in the prompt, an injection that manipulates the model's reasoning still cannot manipulate the authorisation decision.

Bounded agent loop (FR64). The loop caps at four tool-calling iterations, after which it returns a response asking the user to narrow the question. This bounds both cost and the scope of any single interaction.

Mandatory verified authentication (FR65). The assistant endpoint requires a verified token carrying a role claim, because the tool-scoping logic keys off that claim. The shared authentication middleware already rejects unauthenticated requests, so the endpoint's own check is a second line of defence: a request that reached it without a role claim would be refused rather than served with reduced scoping.

Query logging (FR66). Every question is appended to a query log recording the company, employee, role, question, and tools invoked, providing both an operational usage record and evidence of correct scoping behaviour.

Figure 4.11 shows the assistant answering a question within the caller's entitlement and, beside it, declining one outside it. The refusal is produced by the tool's own authorisation check rather than by the model's judgement, which is the distinction Section 3.8.6 turns on.

{FIGURE PLACEHOLDER — Figure 4.11: Two screenshots of the assistant interface. Left: a question the caller is entitled to ask, with the answer returned. Right: a question requesting data outside the caller's entitlement, showing the scoped refusal relayed to the user. Where possible, show the same question asked by two different roles to make the scoping behaviour explicit. Caption goes BELOW the figure.}

Figure 4.11: AI assistant showing (a) a scoped answer and (b) an authorisation refusal

4.4 System Testing

4.4.1 Testing strategy

The backend is tested with Vitest, with the test directory structure mirroring the source structure one-for-one. Three testing patterns are used consistently.

Service unit tests. The database client is mocked as a chainable stub, with query-builder methods returning the stub and terminal calls returning fixed results, and injected directly into the service under test. Tests assert both the returned value and that the correct database calls were made. The most important assertion pattern in this category concerns ordering: where a service performs an authorisation check before a write, the test verifies that an unauthorised call returns an error **without ever invoking the write**, rather than merely verifying that it returns an error. A service that wrote first and checked afterward would pass a naive assertion and fail this one.

Controller unit tests. The service module is mocked and the controller function is invoked directly against a hand-built mock request context, asserting that the service was called with the correct arguments and that the response payload and status are correct.

Integration tests. The real application is exercised end-to-end through the framework's request method with token verification mocked, validating routing and authorisation behaviour through the full middleware chain rather than any single layer in isolation. This is what verifies that the request pipeline specified in Section 3.5.4 actually short-circuits where it should.

Frontend component tests. The React application is tested with Vitest in a simulated browser environment (jsdom) using React Testing Library. Each test renders a feature view with the API client module mocked, interacts with it as a user would by locating elements through their accessible roles and visible text, and asserts both on what the view displays and on the API calls it issues. Animation and charting libraries whose behaviour depends on real layout are replaced with lightweight stubs, so the tests exercise the view's own logic, namely data shaping, conditional rendering, and form handling, rather than third-party rendering code.

New queries are expected to be tested against empty results, not-found cases, and cross-tenant access attempts specifically, on the basis set out in Section 3.8.3 that a tenant-scoping defect is a security failure rather than a correctness one.

4.4.2 Test results

Both suites were executed against the final implementation on 7 October 2026. The results are reported below.

Table 4.2: Automated test suite execution results

Metric			Backend (API)	Frontend (UI)
Test files executed			63	49
Test files passed			63	49
Test files failed			0	0
Individual	test	cases	404	507
executed				
Individual test cases passed			404	507
Individual test cases failed			0	0
Pass rate			100%	100%
Total wall-clock duration			6.17 s	36.04 s

All 112 test files and all 911 individual test cases passed. The backend suite completed in 6.17 seconds of wall-clock time, of which the tests themselves accounted for 1.35 seconds. That figure is a property of the architecture rather than an incidental detail: because the service layer is testable in isolation from both the database and the HTTP layer, as NFR15 requires, no backend test provisions a database, starts a server, or waits on a network call,

and running the full suite is a routine step during development rather than an occasional exercise. The frontend suite is slower, at 36.04 seconds, and the difference lies in set-up rather than in the tests: each test file constructs its own simulated browser environment, and Vitest attributes roughly 30% of the suite's tracked time to that construction alone.

Coverage boundary. Each suite tests its own layer in isolation: the backend suite replaces the database with a mock, and the frontend suite replaces the API client with a mock. Neither exercises the deployed platform, and no end-to-end test drives the user interface against a running backend. The 100% pass rate should be read with that boundary in mind. Section 4.5.3 reports a defect that passed both suites and surfaced only against the real database, which is the clearest evidence of what the boundary leaves untested.

Figure 4.12 shows the terminal output of both suites executed against the final implementation, from which the figures in Table 4.2 are taken.

{FIGURE PLACEHOLDER — Figure 4.12: A screenshot of the terminal after running both suites in sequence from the project root, in a window of about 110 columns, with the end of the output captured so that both "Test Files ... passed" summary blocks are visible together. Use a light terminal theme so it reproduces legibly in print. Caption goes BELOW the figure.}

Figure 4.12: Terminal output of the backend and frontend test suites

4.4.3 Security-relevant testing

Three categories of security-relevant behaviour were tested explicitly rather than assumed.

Cross-tenant access. Tests attempt to retrieve records belonging to one company through a context authenticated against another, asserting that no rows are returned. This is the test category that matters most under the pool isolation model, because it is the only direct verification that the shared data-access wrapper described in Section 4.3.1 is doing what the design claims.

Authorisation ordering. Tests verify that an unauthorised action returns an error without the corresponding write having been attempted, as described in Section 4.4.1. A manager acting on an employee outside their direct-report set must fail before any mutation occurs, not after.

Middleware short-circuiting. Integration tests exercise the full middleware chain to confirm that a request failing authentication, tenant resolution, or role gating terminates before reaching module logic, verifying the pipeline behaviour specified in Section 3.5.4. These include a request that presents a company identifier in a header but no token, which is rejected at the authentication stage, confirming that the fallback described in Section 3.8.7 is no longer accepted.

These tests verify that the controls described in Chapter Three behave as designed. Consistent with Section 3.8.8, they do not constitute a security assessment. No penetration test or third-party audit was conducted.

4.5 Discussion of Results

This section evaluates the system against the objectives in Section 1.3, addressing Objective 8 through response-time measurement and cost-model analysis, and reporting the platform constraints the evaluation surfaced.

4.5.1 Response-time evaluation

Method. Response times were measured against the deployed production Worker rather than the local emulation environment, because cold-start behaviour and the network path are properties of the deployed edge runtime and are not reproduced locally. To keep benchmark traffic out of live tenant data, the measurement script registered a dedicated benchmark company and seeded it with fifty active, payroll-ready employees. The administrator acted as line manager for the other forty-nine, and ten pending leave requests populated the approvals queue. All requests were issued sequentially from a single client in Lagos over the public internet. The deployment ran on Cloudflare's Workers Free plan. The Free and Paid plans differ in their limits and billing rather than in the runtime that executes the code, so the latencies reported here apply to either plan, while the cost model in Section 4.5.2 uses Paid-plan rates because that is the plan a production deployment would require. The plan does matter for one limit, discussed in Section 4.5.3: the CPU time allowed per request.

For each of the ten operations in Table 4.3, the script left the Worker idle for five minutes and then issued one request on a new connection. This is the cold-start sample. It then issued twenty further requests on a kept-alive connection, ten for the assistant query in order to bound inference cost, from which the median and 95th-percentile response times are reported, with percentiles computed by linear interpolation between closest ranks. Write operations were reversed immediately after each timed request, by clocking out after each clock-in and rejecting each submitted payroll run, so that every sample began from the same state. The reversals were not timed.

An idle period makes eviction of the Worker's isolate likely but does not guarantee it, so cold samples were verified rather than assumed. The Worker was instrumented to report, with every response, how many requests the isolate serving it had handled, and a sample is reported as a cold start only where that count was one. Where an attempt landed on an isolate that was still warm, or failed at the network level, the idle period was repeated, up to three attempts in total. Requests that received no response within 60 seconds were recorded as failures and excluded from the statistics. The same instrumentation reports the time each request spent inside the Worker, which allows platform-side latency to be separated from network and connection set-up time in the discussion that follows.

Endpoints were selected to span the two categories the architecture treats differently: interactive reads that a user waits on, and the payroll computation that constitutes the system's heaviest single operation.

Table 4.3: Response-time measurements for representative operations (measured 8 October 2026, Workers Free plan, client in Lagos)

Operation	Category	Cold start (ms)	Warm p50 (ms)	Warm p95 (ms)	Sample size
Authenticate and issue token	Interactive (authentication)	600	251	295	1 cold + 20 warm
Retrieve employee directory (50 records)	Interactive read	885	245	311	1 cold + 20 warm
Retrieve single employee profile	Interactive read	633	221	242	1 cold + 20 warm
Retrieve own leave balance	Interactive read	664	237	250	1 cold + 20 warm
Submit leave request	Interactive write	707	271	367	1 cold + 20 warm
Record attendance clock-in	Interactive write	679 (see note)	247	309	1 cold + 20 warm
Retrieve manager pending-approvals queue	Interactive read	616	198	236	1 cold + 20 warm
Preview payroll run (50 employees)	Batch computation	871	303	319	1 cold + 20 warm
Submit payroll run (50 employees)	Batch computation	1,013	415	486	1 cold + 20 warm

Assistant query (single tool call)	Interactive read (AI)	3,221	1,909	2,447	1 cold + 10 warm
---------------------------------------	--------------------------	-------	-------	-------	---------------------

Note: all cold samples were confirmed as served by a newly created isolate. The attendance clock-in cold sample was retaken under the same procedure after the first attempt was delayed by a transient client-side connection stall, as discussed below.

Warm-path interactive operations completed with medians between 198 and 271 milliseconds and 95th percentiles no higher than 367 milliseconds, and none of the 140 warm samples taken for the seven interactive operations other than the assistant exceeded 400 milliseconds. NFR2 deliberately left the acceptable bound to measurement rather than fixing it in advance, so a reference point is needed. The response-time limits long used in usability engineering treat one second as the threshold below which a delay, though noticeable, does not interrupt the user's flow of thought (Nielsen, 1993). Every warm interactive operation falls well inside that threshold, and NFR2 is satisfied. The composition of these figures matters as much as their size. The time spent inside the Worker accounted for only 21 to 81 milliseconds of each of these warm interactive requests; the remaining 70 to 92 per cent was network transit between the client and the platform. At this scale, response time is governed by the network path rather than by execution, which is precisely the condition Raith et al. (2023) identify as the motivation for moving serverless execution to the edge.

That observation is qualified by where the requests were actually served. Although the platform identified the test client as located in Nigeria, every request was answered by Cloudflare's Amsterdam data centre rather than one in Lagos, as shown by the data-centre identifier the platform attaches to each response, and the network round trip to that data centre was approximately 140 milliseconds. The cause of this routing was not established. It may reflect the client's internet service provider's interconnection arrangements or the platform's own routing decisions, and neither could be verified within this project. The database, meanwhile, is hosted in Western Europe. The edge-proximity benefit argued for in Chapter Two was therefore not realised for a Nigerian client in this deployment. Serving the Worker from Lagos would not by itself remove the distance, however, because each database query would then make the journey to Europe instead, and a single request issues several queries. For a deployment serving Nigerian users, the location of the database relative to those users is a design variable in its own right, and Chapter Five returns to it.

The cold-start samples show a larger penalty in aggregate than in substance. The first request after idleness took between 600 and 885 milliseconds for the interactive operations, roughly three times their warm medians, yet most of that difference did not come from the platform starting the Worker. Between 326 and 546 milliseconds of each cold sample was spent establishing a new encrypted connection before the request was sent, a cost any architecture incurs on a client's first request and one the warm samples, which reused a connection, did not pay. Once the connection was open, the cold requests took between 11 and 100 milliseconds longer than the corresponding warm medians, and the time inside the Worker exceeded its warm figure by between 23 and 36 milliseconds. The penalty attributable to isolate start-up is therefore in the order of tens of milliseconds, and even the

complete cold figures remain below the one-second threshold, so on this evidence cold starts are not material for the interactive category. Two further observations are consistent with that conclusion. For two of the ten operations, the first request after five minutes of idleness was still served by a warm isolate, which suggests that under light use a user may not meet a cold start at all. And the first cold sample for the attendance clock-in, at 6,888 milliseconds, was spent almost entirely, 6,545 milliseconds of it, establishing the connection, with only 103 milliseconds inside the Worker. It was attributed to a transient delay on the client's network and retaken; the retaken value appears in Table 4.3.

The payroll operations completed comfortably at the tested size. Previewing a fifty-employee run took a median of 303 milliseconds and submitting one took 415 milliseconds, with the first submission after idleness at 1,013 milliseconds. Submission is the slowest operation other than the assistant because it persists the run header and fifty payslips, written as seventeen insert statements that each respect D1's per-statement parameter limit and are committed with the header as a single atomic batch. All forty-two payroll requests, twenty-one previews and twenty-one submissions, succeeded on the Free plan, and none was rejected for exceeding its CPU allowance of 10 milliseconds per request. Elapsed time is not the binding constraint. Because the platform's CPU clock advances only while the code is executing rather than while it waits on input or output, the 173 milliseconds a warm submission spent inside the Worker is largely time spent waiting on the database, which does not count towards the CPU allowance. The implications for larger companies are taken up in Section 4.5.3.

The assistant query departs from these patterns and is reported separately for that reason. Its warm median was 1,909 milliseconds and its 95th percentile 2,447 milliseconds, with about 90 per cent of that time spent inside the Worker, overwhelmingly in model inference across the two model calls that a single tool invocation requires. This places it above the one-second threshold but well within the ten-second limit beyond which users' attention is lost (Nielsen, 1993). For a conversational feature, where a short wait is expected, this is acceptable, and the interface shows a progress indicator for the duration of the request.

Figure 4.13 plots these measurements, grouping the interactive operations separately from the batch payroll operations so that the difference in both magnitude and cold-start sensitivity between the two categories is readable at a glance.

{FIGURE — Insert the generated chart from project/figures/figure_4_2_response_times.png. Caption goes BELOW the figure.}

Figure 4.13: Cold-start and warm-path response times by operation category

Source: measurements reported in Table 4.3.

A note on the comparative claim flagged in Section 2.5. That section recorded that the assertion of lower cold-start latency for isolate-based platforms relative to container-based ones is drawn from vendor and practitioner sources rather than from peer-reviewed literature, and that this project's measurement is positioned to speak to it. The measurements above are single-platform and therefore cannot settle the comparison; they establish what this platform delivers for this workload, not how it ranks against alternatives. That limitation is stated rather than elided.

4.5.2 Cost-model analysis

The second component of Objective 8 is an analytical comparison of the platform's usage-based cost model against the fixed-cost profile of a provisioned deployment. Unlike the response-time evaluation, this can be computed directly from published rates.

Rate card. All figures were re-verified against Cloudflare's published pricing documentation on 7 October 2026.

Table 4.4: Platform rate card (Workers Paid plan)

Service	Included in plan	Overage rate
Workers subscription	—	\$5.00 / month base
Worker requests	10 million / month	\$0.30 / million
Worker CPU time	30 million CPU-ms / month	\$0.02 / million CPU-ms
D1 rows read	25 billion / month	\$0.001 / million rows
D1 rows written	50 million / month	\$1.00 / million rows
D1 storage	5 GB	\$0.75 / GB-month
R2 storage	10 GB-month	\$0.015 / GB-month
R2 Class A operations (writes)	1 million / month	\$4.50 / million
R2 Class B operations (reads)	10 million / month	\$0.36 / million

R2 egress	Unlimited	\$0.00
-----------	-----------	--------

Pages static asset requests	Unlimited	\$0.00
-----------------------------	-----------	--------

Modelling assumptions. Three scenarios are modelled at increasing tenant scale. Each assumes 40 API requests per employee per working day across 22 working days per month, an average of 5 CPU-milliseconds per request, approximately 30 database rows read per request, writes on 15% of requests at approximately 2 rows each, approximately 160 KB of relational data per employee in D1, and 4 stored documents per employee averaging 500 KB. These assumptions are stated so the model can be recomputed under different ones.

Table 4.5: Modelled monthly platform cost at three scales

		Small (10 tenants, 500 employees)		Medium (100 tenants, 5,000 employees)		Large (1,000 tenants, 50,000 employees)	
Monthly requests	API	0.44 million		4.4 million		44 million	
Worker charge	request	\$0.00	(within allowance)	\$0.00	(within allowance)	\$10.20	
Worker charge	CPU	\$0.00	(within allowance)	\$0.00	(within allowance)	\$3.80	
D1 rows read / charge		13.2 million / \$0.00		132 million / \$0.00		1.32 billion / \$0.00	
D1 rows written / charge		0.13 million / \$0.00		1.32 million / \$0.00		13.2 million / \$0.00	
D1 storage charge		\$0.00	(within allowance)	\$0.00	(within allowance)	\$2.25	

R2 storage charge	/	1 GB / \$0.00		10 GB / \$0.00		100 GB / \$1.35	
R2 operations charge		\$0.00 (within allowance)		\$0.00 (within allowance)		\$0.00 (within allowance)	
Subscription		\$5.00		\$5.00		\$5.00	
Total cost	monthly	\$5.00		\$5.00		\$22.60	
Cost per employee month	per per	\$0.010		\$0.001		\$0.0005	

Inference costs for the assistant module are billed on a separate metric and are excluded from this model, since usage depends entirely on how heavily the assistant is adopted and cannot be estimated from headcount. This exclusion is stated rather than absorbed into the totals.

Three findings follow.

First, cost is flat across two orders of magnitude of tenant growth. A hundredfold increase in employees, from 500 to 50,000, raises the monthly cost by a factor of roughly 4.5, and the first tenfold increase raises it not at all. This is the direct consequence of usage-based billing with substantial included allowances: the platform absorbs the entire small and medium scenario within the base subscription.

Second, and more consequentially for the argument made in Chapters One and Two, the cost floor at zero activity is the subscription alone. A provisioned deployment's floor is the sum of its running instances, incurred identically whether the system serves one request or a million. Section 1.2 identified this as the core cost inefficiency of conventional cloud HRIS, and the model above is the direct measurement of its absence: outside a payroll run, an HR system serving a small organisation is close to idle, and here that idleness costs nothing beyond the subscription.

Third, the cost structure inverts the usual per-seat economics. Cost per employee **falls** as the platform grows, from one cent per employee per month at the small scale to a twentieth of a cent at the large scale, because fixed allowances are amortised across more users. Commercial platforms priced per employee per month exhibit the opposite behaviour.

Comparison with a provisioned baseline. A functionally equivalent provisioned deployment requires, at minimum, an application server, a managed relational database, and object storage, each running continuously. To give Figure 4.14 a concrete reference line, an indicative baseline is taken from DigitalOcean's published list prices: a 2 GB virtual machine at \$12.00 per month, the smallest single-node managed PostgreSQL instance at \$15.15, and object storage at \$5.00, a combined \$32.15 per month before any traffic is served (DigitalOcean, 2026a, 2026b, 2026c). The step to \$44.15 at 20 million requests per month represents the addition of a second application instance. Where that step falls depends on the workload, and it is illustrative rather than measured. The comparison is not offered as a ranking of vendors, which would not generalise beyond one provider at one moment. It is offered to show the shape of the two cost functions: a provisioned deployment's cost depends on provisioned capacity and elapsed time, whereas this system's cost depends on the work actually performed. At the small scale the system performs so little work that its cost is the subscription alone, while the provisioned equivalent pays for a full month of continuously running capacity to serve fewer than half a million requests.

{FIGURE PLACEHOLDER — Figure 4.14: A line chart with monthly request volume on the horizontal axis (logarithmic scale, from 0 to 100 million) and monthly cost on the vertical axis. Plot two lines: (1) this system's usage-based cost, which is flat at \$5 until the included allowances are exhausted and then rises gently; (2) an indicative provisioned deployment, drawn as a horizontal line at a stated fixed monthly cost with a step increase where an additional instance would be required. Mark the three modelled scenarios as points on the first line. The visual point is that one line starts near zero and rises, while the other starts high and steps. Caption goes BELOW the figure.}

Figure 4.14: Monthly platform cost against request volume, usage-based versus provisioned

Source: usage-based cost computed from the rates in Table 4.4 and the assumptions in Table 4.5. Provisioned baseline from DigitalOcean list prices (DigitalOcean, 2026a, 2026b, 2026c).

Figure 4.14 also shows where the comparison stops favouring the usage-based model. Beyond approximately 82 million requests per month, or about 93,000 employees under the stated assumptions, the modelled usage-based cost exceeds the two-instance provisioned baseline, driven by per-request and CPU-time charges. Two qualifications limit what this crossover means. First, the provisioned line is optimistic at that volume, since a single small database and two application instances sized for the small scale would be unlikely to sustain that load without further capacity steps. Second, and more decisively, the dotted line marks the point, at about 55 million requests or 62,500 employees, beyond which the pool model's single database would exceed D1's 10 GB limit. The upper end of the curve therefore already assumes the per-tenant database architecture discussed in Section 4.5.3. Across the range this project targets, the usage-based model is the cheaper of the two by a wide margin.

4.5.3 Platform constraints identified

The evaluation surfaced one constraint that materially qualifies the tenant-isolation decision taken in Section 3.6.1, and it is reported here because an examiner will find it and because it is a genuine finding rather than a defect.

Cloudflare's published limits for D1 specify a **maximum database size of 10 GB** on the paid plan, which the documentation states explicitly cannot be increased, and record that **each individual D1 database is inherently single-threaded and processes queries one at a time**, so that maximum throughput is determined directly by average query duration (Cloudflare, 2026d). The documentation quantifies this: an average query duration of 1 ms permits approximately 1,000 queries per second, while an average of 100 ms permits approximately 10 per second. A database receiving more concurrent requests than it can service queues them, and returns an overload error once that queue is full.

The same documentation states that D1 is designed for horizontal scale-out across multiple smaller databases on a per-user, per-tenant, or per-entity basis, and that an account may hold up to 50,000 databases on the paid plan at no additional cost, increasable on request into the millions, since billing is based on query and storage volume rather than on database count (Cloudflare, 2026d).

The implication for the pool model is direct. Placing every tenant in one shared database means every tenant's queries contend for one single-threaded database, and the aggregate data of all tenants must fit within one 10 GB ceiling. Neither constraint binds at the small or medium scales modelled in Section 4.5.2. Both would bind well before the large scenario: 50,000 employees with accumulated payroll, attendance, and recruitment history would approach the storage ceiling, and the concurrency characteristics of a single-threaded database shared across a thousand tenants during a simultaneous month-end period would become the limiting factor on throughput long before request cost did.

This does not invalidate the pool decision, and it is worth being precise about why. The reasons given in Section 3.6.1, atomic single migrations, tenant onboarding without provisioning, negligible fixed cost per tenant, and single-query cross-tenant reporting, all hold, and they are the right priorities for a system at this project's scale and timeline. What the constraint establishes is that the pool model has a **discoverable ceiling on this platform specifically**, and that the platform's own design guidance points toward the silo model beyond it. That is a more useful finding than either an unqualified endorsement or a retrospective claim that silo should have been chosen from the outset: the correct architecture is scale-dependent, and this evaluation identifies approximately where the crossover lies. Section 5.4 carries this forward as the principal item of future work.

A second constraint bounds the size of a payroll run that a single Worker invocation can process. The binding limit is CPU time rather than elapsed time, since time spent waiting on the database does not count towards it. The limit depends on the plan: an HTTP request on the Free plan, on which this evaluation ran, may consume 10 milliseconds of CPU, while on the Paid plan the default is 30 seconds, configurable to a maximum of five minutes (Cloudflare, 2026h). Each isolate is also limited to 128 MB of memory on both plans. The benchmark reported in Section 4.5.1 establishes an upper bound on this figure without reading it directly. All forty-two payroll requests against a fifty-employee company succeeded on the Free plan, and none returned error 1102, the code the platform issues when a Worker

exceeds its resource limits (Cloudflare, 2026h). Consumed CPU time for a fifty-employee run was therefore below 10 milliseconds, which leaves considerable headroom against the Paid plan's 30-second default. {INSERT IF READ FROM WORKERS LOGS: the exact CPU time for the fifty-employee submission, and the headcount each plan's limit implies if CPU time scales linearly with employees.} Beyond that size the run would need to be divided across invocations, for example by queueing employees in batches. This was anticipated in Section 3.2.3 and is recorded as the second scaling boundary of the current implementation.

A third constraint was found by the evaluation itself rather than anticipated. The first attempt to submit a fifty-employee payroll run against the real database failed. D1 limits a single SQL statement to 100 bound parameters (Cloudflare, 2026d), and the ORM binds a value for every column of the payslip table, twenty-seven in all, for each row it inserts. The implementation already divided the payslip insert into chunks, but the chunk size of four rows had been chosen from the number of fields the code supplied rather than the number of columns the ORM binds, so each chunk required 108 parameters. Because the run header was written before the payslips, the failure also left a header without payslips, which blocked any resubmission for that period. The defect was corrected by deriving the chunk size from the table's column count and by writing the header and every payslip chunk in a single atomic batch, after which the submissions reported in Table 4.3 succeeded. Both test suites had passed throughout. A mocked database accepts any statement, and the parameter limit exists only in the real platform, so this is a concrete instance of the coverage boundary noted in Section 4.4.2. It also supports the recommendation in Chapter Five for integration tests that run against a real D1 instance.

4.5.4 Evaluation against the objectives

Table 4.6: Achievement of objectives

Objective	Status	Evidence
1. Review the literature	Achieved	Chapter Two; gap established in Section 2.12
2. Design data model and tenant isolation strategy	Achieved	Sections 3.6.1–3.6.7; implemented per Section 4.3.1; constraint reported in Section 4.5.3
3. Token authentication and layered authorisation	Achieved	Sections 4.3.1 and 4.4.3; earlier deviation resolved (Section 3.8.7)

4. Core transactional HR modules	Achieved	Sections 4.3.2–4.3.5
5. Statutory payroll engine	Achieved	Section 4.3.7; computation sequence in Section 3.6.7
6. Extended HR modules	Achieved, integrations implemented with partially	Sections 4.3.6, 4.3.8–4.3.10; integration limitation stated in Section 4.3.10
7. Role-scoped assistant	AI Achieved	Section 4.3.11
8. Evaluation of performance and cost	Achieved	Sections 4.4.2, 4.5.1 and 4.5.2; single-platform, single-client measurement acknowledged in Section 4.5.1

Objective 6's integration catalogue is partially implemented, as stated, and the deviation that previously qualified Objective 3 has been resolved (Section 3.8.7). Objective 8 is discharged empirically for response time, from measurements against the deployed system, and analytically for cost, with the single-platform and single-client limits of the measurement stated in Section 4.5.1.

The aim stated in Section 1.3 was to build a scalable multi-tenant HRMS with an integrated statutory payroll engine on serverless architecture, eliminating fixed infrastructure cost and operational overhead while maintaining tenant isolation, role-scoped access, and auditable compliance. On the evidence of this chapter: the fixed infrastructure cost is demonstrably absent, since the production environment has no hardware specification and the cost floor at zero activity is a five-dollar subscription; the operational overhead is absent, since no instance is sized, patched, or scaled; tenant isolation and role scoping are implemented and tested; and statutory computation is auditable, since every deduction is stored decomposed and every remittance obligation is traceable to the run that generated it. The scalability claim holds within the bounds identified in Section 4.5.3, which is a narrower claim than the aim asserted and is stated as such.